Python Programming for Green Skilling and Sustainability

HANDBOOK

Table of Contents

Chapter 3: Python Programming for Green Skilling and Sustainability 2

3.1 Introduction 2

3.2 Installation and Setting Up Path 6

3.3 Python Variable and Identifiers 14

3.4 Data Types and Operators 16

3.5 Control Structures 44

3.6 Functions in Python 49

3.7 Data Manipulation and Analysis: Working with Data 52

3.8 Pandas Library 53

3.9 NumPy Library 62

3.10 Matplotlib 66

3.11 Seaborn 71

References: 75

Chapter 3: Python Programming for Green Skilling and Sustainability

Learning Outcomes:

By the end of this chapter, students will be able to:

- Define Python and identify its key features and advantages.
- Recall the steps required for Python installation and setting up a development environment.
- Describe the purpose of variables and different data types (integers, floats, strings, booleans).
- Identify the purpose of Pandas for data manipulation with data frames and series.
- Recognize NumPy as a library for handling arrays and numerical data.
- Describe the function of Matplotlib in creating visualizations like plots and charts.
- Explain Seaborn's role in creating advanced visualizations and statistical plots.

3.1 Introduction

Python is a high-level, interpreted, interactive, and object-oriented scripting language designed with readability and simplicity in mind. Unlike many programming languages that rely heavily on punctuation, Python uses English keywords more frequently, resulting in fewer syntactic constructions and a more intuitive coding experience.

3.1.1 Key Features of Python

Interpreted

Python code is processed at runtime by the interpreter, eliminating the need for a separate compilation step before execution. This feature is similar to languages such as Perl and PHP.

Interactive

Python allows for direct interaction with the interpreter, which facilitates real-time code writing and testing. This interactive nature is valuable for rapid prototyping and experimentation.

Beginner-Friendly

Python is renowned for being an excellent choice for beginners. Its simple syntax and readability support the development of various applications, from basic text processing scripts to complex web browsers and games.

Features of Python

Source: https://unstop.com/blog/features-of-python

Easy to Code and Understand

Python's high-level nature and straightforward syntax make it much easier to learn and use compared to other languages like C, C#, JavaScript, and Java. Beginners can often grasp the fundamentals within a few hours or days, thanks to Python's emphasis on readability and simplicity.

Expressive Language

Python is highly expressive, meaning that a single line of Python code can accomplish what might require multiple lines in other languages. This feature enhances productivity and makes the codebase more readable and maintainable.

Free and Open Source

Python is free to use and can be easily installed on any system. As an open-source language, Python's source code is available for anyone to modify and distribute, encouraging community contributions and fostering a robust ecosystem.

Interpreted Language

Unlike compiled languages, Python executes code line by line. This approach simplifies debugging by providing immediate feedback on errors, which is particularly useful for identifying and fixing issues in a complex codebase.

Object-Oriented Language

Python supports object-oriented programming (OOP), enabling developers to create and utilize classes and objects. Key OOP principles such as inheritance, polymorphism, data abstraction, and encapsulation are all well-supported in Python.

Dynamically Typed Programming Language

In Python, the type of a variable is determined at runtime, allowing for greater flexibility in coding. This dynamic typing facilitates rapid development without the need to specify data types explicitly.

Cross-Platform Language

Python is designed to be cross-platform, meaning that code written on one operating system can run on another without modification. This feature simplifies development and testing across different environments.

Large Standard Library

Python comes with a comprehensive standard library that includes pre-written code for many common tasks such as file I/O, unit testing, web browsing, database access, CGI, and image manipulation. This extensive library accelerates development by reducing the need to write code from scratch.

Extensible Language

Python can be extended by integrating with other languages such as C++ or Java. This extensibility allows developers to optimize performance and add functionalities beyond the core Python language.

Platform Independent

Python code is platform-independent, which means it can be executed on various operating systems like Windows, Unix, Linux, and macOS without requiring modifications. This feature ensures seamless development across diverse platforms.

Simple Syntax

Python's syntax is designed to be clear and readable. The language emphasizes simplicity and minimalism, which helps reduce the learning curve for new programmers and facilitates easier code maintenance.

High-Level Language

As a high-level language, Python abstracts away much of the complexity associated with system architecture and memory management. This abstraction allows developers to focus on writing code rather than handling low-level system details.

Rich Library Support

Python boasts an extensive collection of libraries and frameworks, which provide pre-written solutions for a wide range of tasks. This rich library support helps accelerate development and minimizes the need for custom implementations.

History of Python

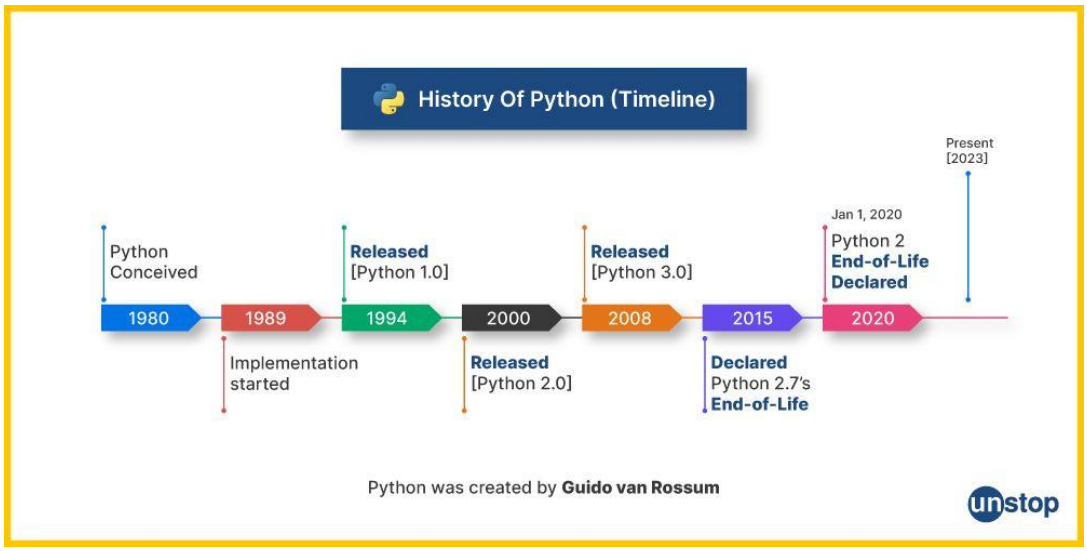
Python Timeline/History and IEEE Rankings

Python is a widely-used general-purpose, high-level programming language with a rich history of development and evolution.

Key Milestones in Python's History

- **1980**: Python was conceived.
- **1989**: Implementation of Python began.
- **1991**: Guido van Rossum published the first version of Python (0.9.0) in February at CWI (Centrum Wiskunde & Informatica) in Amsterdam, Netherlands.
- **1994**: Python 1.0 was officially released.
- **2000**: Python 2.0 was released.
- **2008**: Python 3.0 was introduced.
- **2020**: Python 2.7 reached its end-of-life.
- **2024**: Present year.

Python's development has witnessed several key milestones, each marking significant advancements in its features and usability.



Source: https://unstop.com/blog/what-is-python

Notable Facts

- **ABC Influence**: Python draws inspiration from the ABC Programming Language, which was developed at CWI.
- **Naming**: Guido van Rossum chose the name "Python" in honor of his admiration for Monty Python's Flying Circus.
- **Development Team**: Python is maintained by a core development team, with Guido van Rossum continuing to influence its direction.

IEEE Rankings

The latest rankings from IEEE Spectrum reflect Python's strong presence across various domains. The report analyzes data from diverse sources to assess programming language popularity based on different use cases, including web development, enterprise applications, mobile development, and embedded systems.

Ranking Metrics

- **Social Channels**: The rankings consider the language's popularity on platforms like Twitter, StackOverflow, GitHub, and Reddit.

- **Job Market**: The rankings also account for the demand for languages in the job market.

Python consistently ranks highly in these metrics, underscoring its widespread adoption and ongoing relevance in the programming world.

3.2 Installation and Setting Up Path

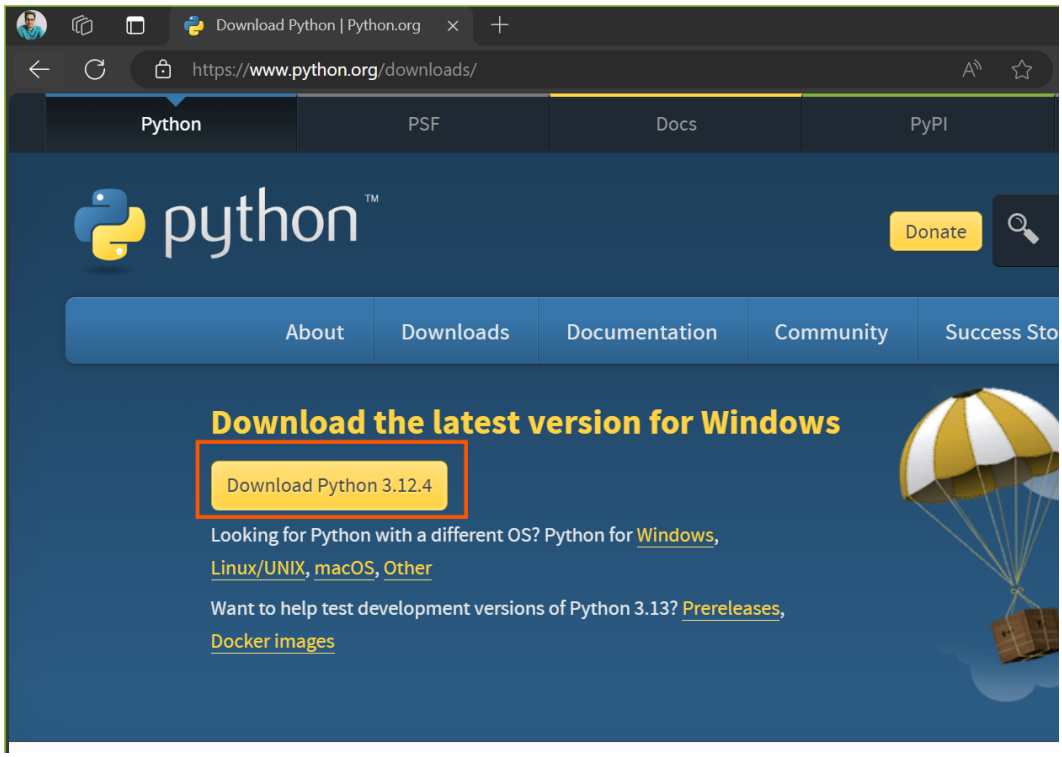
3.2.1 Python Installation for Windows

List of Hardware/Software Requirements

1. Laptop/Computer with Windows OS
2. Python Software Installation

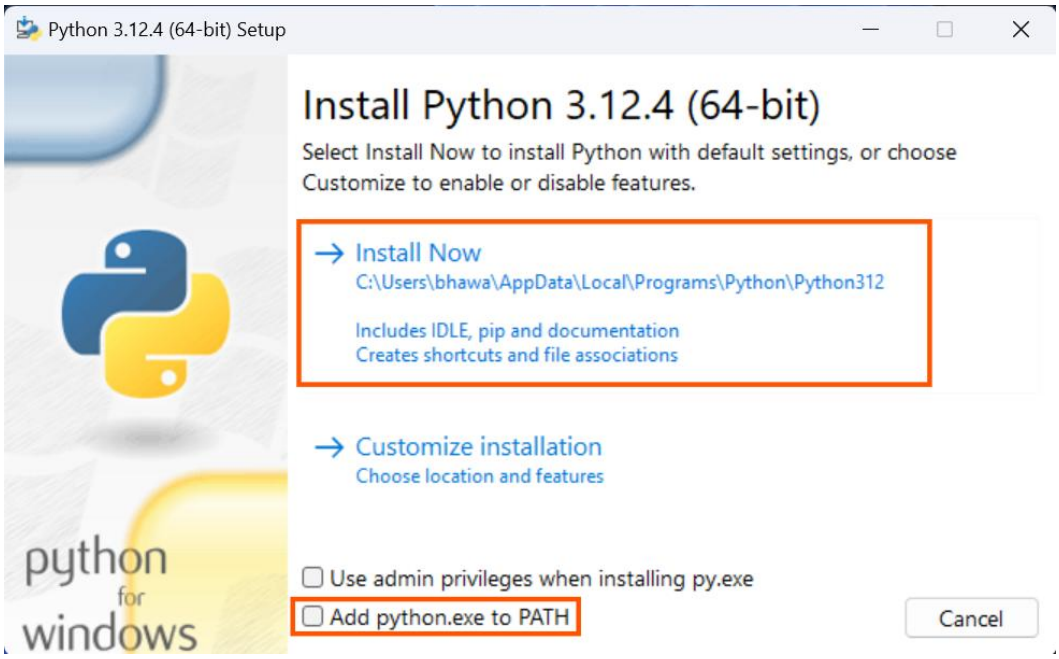
Steps: Install Python Software on Windows

1. Open your web browser and navigate to python.org/downloads.



2. Click on the download link suitable for your system:
   - Choose either Windows x86-64 executable installer for 64-bit or Windows x86 executable installer for 32-bit.

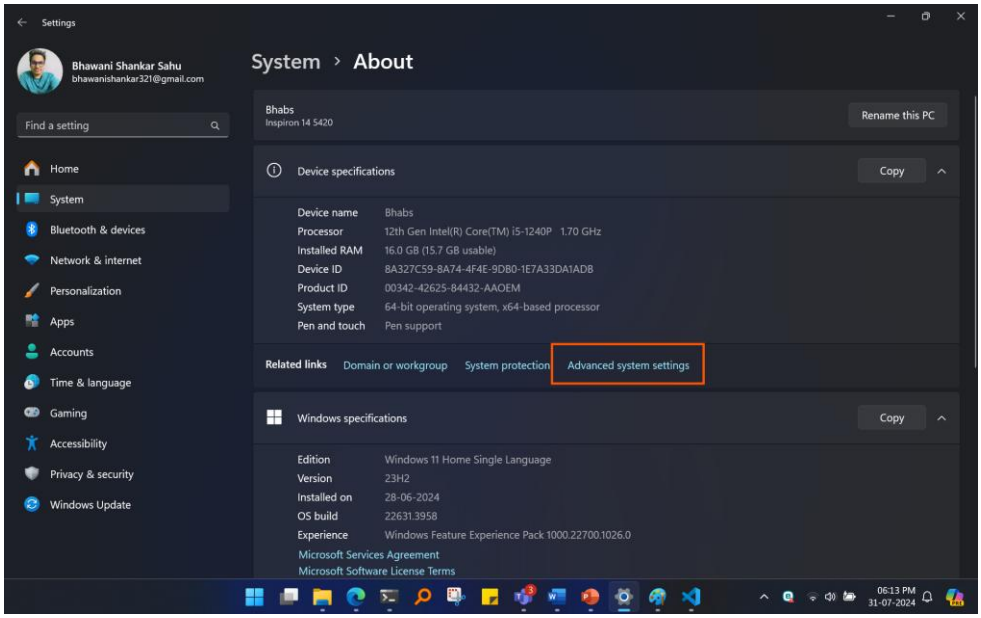
3. After downloading the file, launch the setup and click on “Add python.exe to Path“ and then click on “Install Now”.



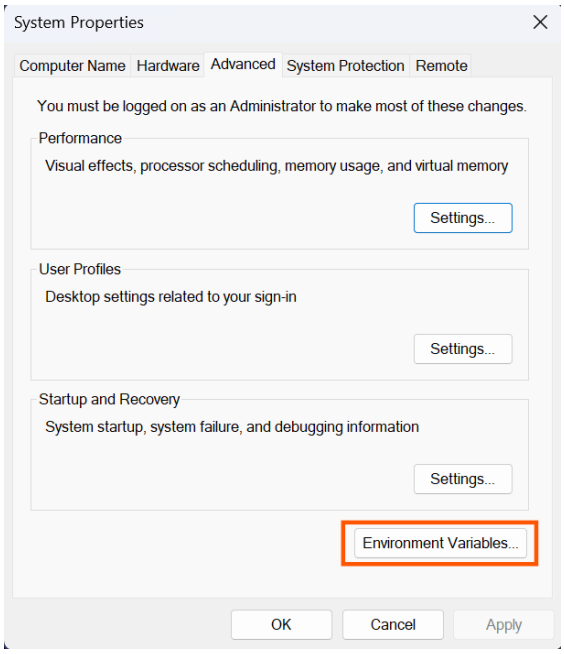
How to Set Python Path in Windows

To permanently modify the default environment variables:

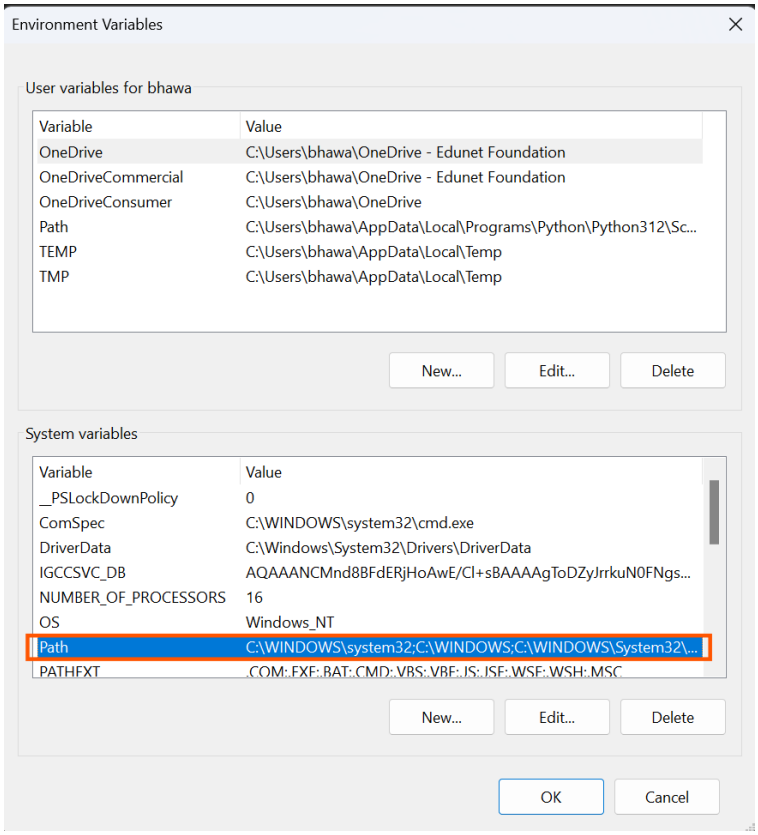
1. **Right-click** 'My Computer' (or 'This PC' in newer versions).
2. Select **'Properties'** from the context menu.
3. Click on **'Advanced system settings'** on the left side.



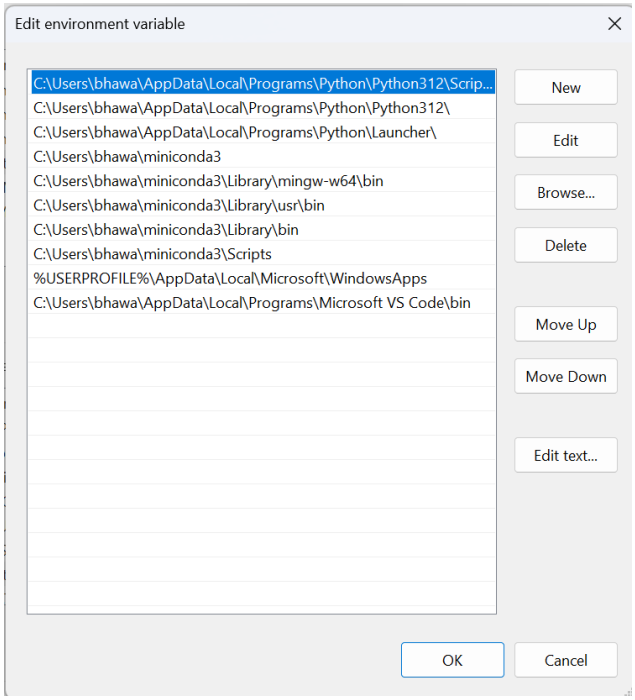
4. In the Advanced tab, click **'Environment Variables...'**.



5. Under 'System Variables', find and select the **'Path'** variable, then click **'Edit'**.

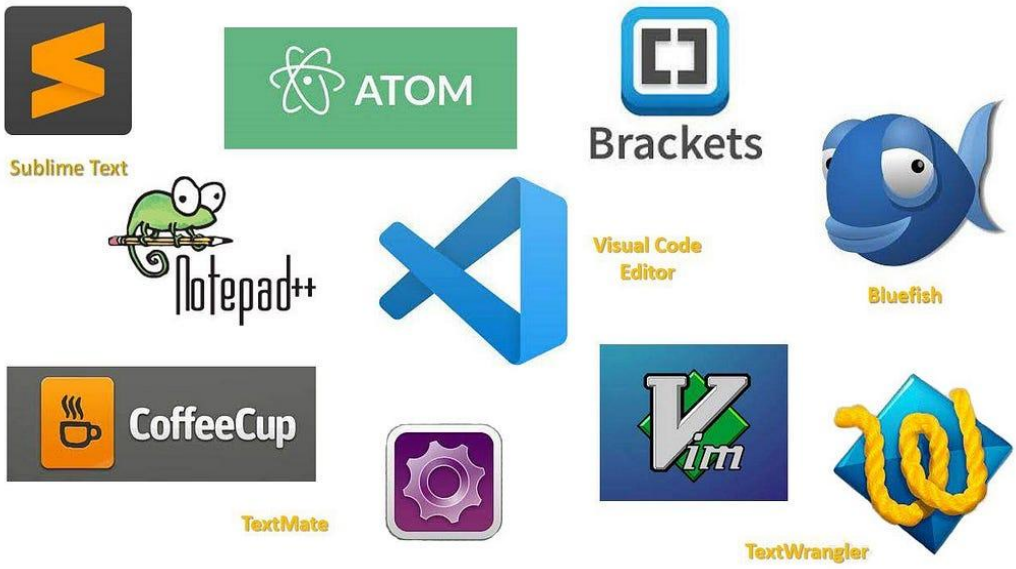


6. Add Python's path to the end of the list (paths are separated by semicolons ;).



3.2.2 IDEs and Code Editors for Python

When developing Python applications, choosing the right development environment can significantly impact productivity and ease of coding. Both IDEs and code editors serve to write and manage code, but they offer different features and levels of complexity.



Source: https://medium.com/@talktorahul.b/top-10-free-code-editor-software-a063f89fbc86

Integrated Development Environments (IDEs)

IDEs are comprehensive tools that provide everything needed for software development in one place. They typically include a code editor, a debugger, build automation tools, and other features.

Features of IDEs

- **Code Editing:** Advanced text editing features like syntax highlighting, code completion, and intelligent code suggestions.
- **Debugger:** Tools for stepping through code, setting breakpoints, and inspecting variables.
- **Build and Run Tools:** Integrated support for running and building your code.
- **Version Control Integration:** Built-in support for Git, Mercurial, or other version control systems.
- **Project Management:** Tools for managing files and directories within projects.
- **Plugins and Extensions:** Support for adding extra functionalities and integrations.

Popular Python IDEs

1. PyCharm

- Developed by JetBrains, PyCharm is a powerful IDE tailored specifically for Python. It offers advanced features such as code analysis, a robust debugger, and support for web development frameworks like Django and Flask.
- Key Features:
    - Smart code completion
    - Advanced refactoring
    - Integrated testing tools
    - Git integration
    - Python-specific project templates

2. Visual Studio Code (VS Code)

- VS Code is a lightweight, open-source code editor developed by Microsoft. With the Python extension, it can provide many features typical of full-fledged IDEs.
- Key Features:
    - IntelliSense code completion
    - Debugging support
    - Integrated terminal
    - Extensions marketplace
    - Git integration

3. Spyder

- Spyder is an open-source IDE designed for scientific computing and data analysis. It integrates well with scientific libraries such as NumPy, SciPy, and Matplotlib.
- Key Features:
    - Variable explorer
    - Integrated IPython console
    - Scientific libraries support
    - Code analysis tools
    - Interactive data visualization

4. Thonny

- Thonny is a beginner-friendly IDE developed for learning and teaching Python. It offers a simple interface and basic debugging tools.
- Key Features:
    - Simple user interface

    - Built-in debugger
    - Easy package management
    - Step-through debugging

Code Editors

Code editors are simpler tools focused on editing code rather than providing a full development environment. They are usually more lightweight compared to IDEs.

Features of Code Editors

- **Syntax Highlighting:** Basic color coding to differentiate code elements.
- **Code Autocompletion:** Suggestions for completing code as you type.
- **File Management:** Tools for managing and navigating code files.
- **Customization:** Support for themes and plugins to enhance functionality.
- **Integration with External Tools:** Limited but useful integration with external tools and terminal.

Popular Python Code Editors

1. Sublime Text

- Sublime Text is a fast, cross-platform code editor known for its speed and simplicity. It supports Python through plugins and packages.
- Key Features:
    - Multiple selections and editing
    - Command palette for quick access
    - Customizable via plugins
    - Syntax highlighting for Python

2. Atom

- Atom, developed by GitHub, is a customizable and open-source code editor. It supports Python through community packages.
- Key Features:
    - Built-in Git and GitHub integration
    - Customizable through packages and themes
    - Real-time collaborative editing with Teletype
    - Flexible and extensible

3. Notepad++

- Notepad++ is a free source code editor that supports multiple languages including Python. It is lightweight and fast.
- Key Features:
    - Syntax highlighting and folding
    - Plugin support for additional features
    - Multi-document editing
    - Regular expression search and replace

4. Jupyter Notebook

- Jupyter Notebook is an open-source web application that allows you to create and share documents that contain live code, equations, visualizations, and narrative text.

- Key Features:
    - Interactive code execution
    - Integration with data visualization libraries
    - Markdown support for rich text
    - Integration with various data science tools

4. Choosing Between IDEs and Code Editors

- **Complexity and Features:** IDEs are suitable for complex projects requiring advanced features and integrations. Code editors are ideal for simpler tasks or when a lightweight tool is preferred.
- **Performance:** Code editors generally perform faster and use less memory, making them suitable for quick edits and smaller projects.
- **Learning Curve:** IDEs often have a steeper learning curve due to their comprehensive feature sets, while code editors are more straightforward and easier to pick up for beginners.

Jupyter Notebook

Jupyter Notebook is an open-source web application that allows you to create and share documents that contain live code, equations, visualizations, and narrative text. It is widely used for data analysis, scientific computing, and machine learning. Anaconda is a popular distribution of Python and R for scientific computing and data science, which includes Jupyter Notebook along with other useful tools and libraries.

Features of Jupyter Notebook

Jupyter Notebook provides an interactive computing environment where you can write and execute code in real-time.

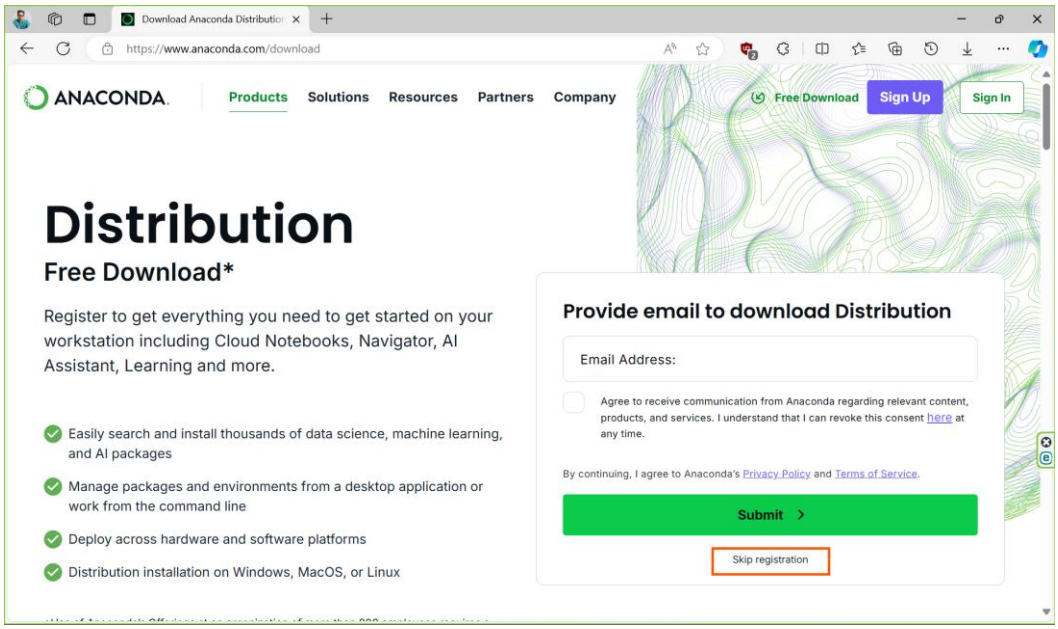
Key features include:

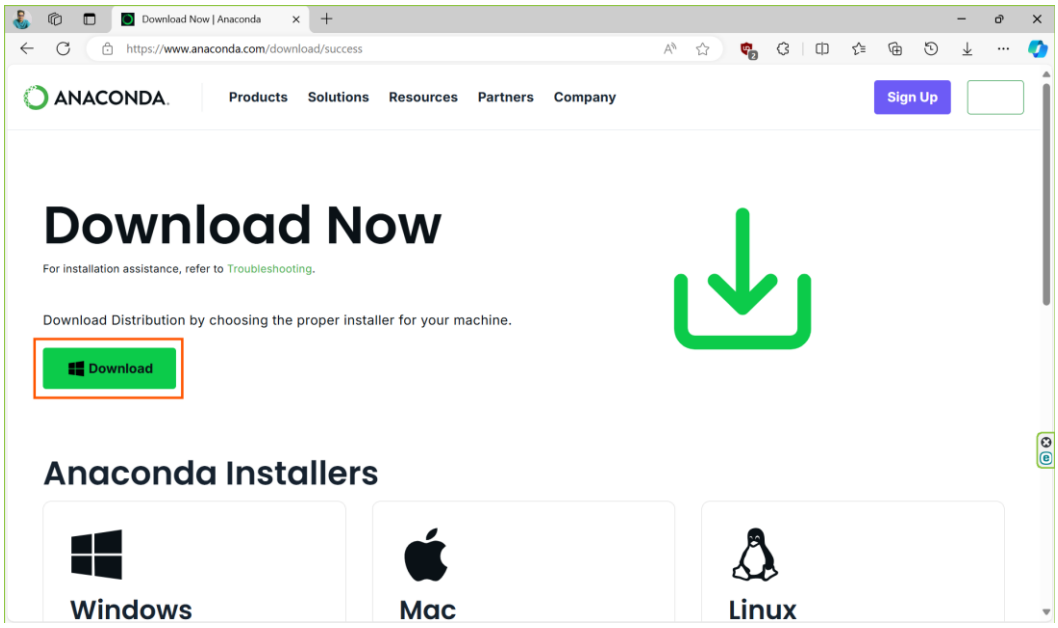
- **Interactive Code Execution:** Run code in small chunks (cells) and see the output immediately.
- **Rich Text Support:** Combine code with narrative text, equations, and visualizations using Markdown and LaTeX.
- **Data Visualization:** Integrate with libraries like Matplotlib, Seaborn, and Plotly to create visualizations.
- **Sharing and Collaboration:** Easily share notebooks via GitHub, email, or JupyterHub.

Setting Up Jupyter Notebook Using Anaconda

Installing Anaconda

1. **Download Anaconda:** Visit the Anaconda Distribution website and download the installer for your operating system (Windows, macOS, or Linux). (Link: Download Anaconda Distribution | Anaconda)





2. **Install Anaconda:** Run the installer and follow the on-screen instructions. Make sure to check the option to add Anaconda to your PATH environment variable.

Launching Jupyter Notebook

1. **Open Anaconda Navigator:** Launch Anaconda Navigator from your start menu or applications folder.
2. **Launch Jupyter Notebook:** In Anaconda Navigator, click the Launch button under the Jupyter Notebook section. This will open a new tab in your default web browser with the Jupyter Notebook interface.

Creating a New Notebook

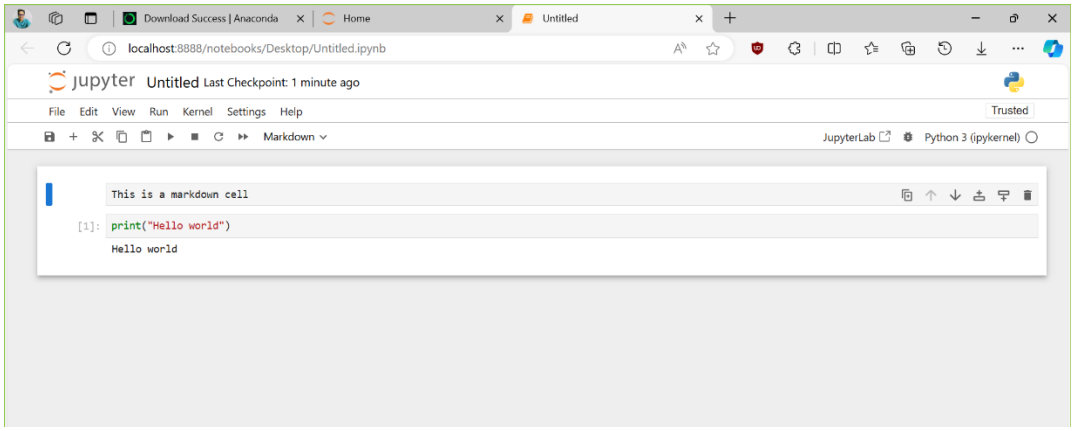
1. **Navigate to the Directory:** Use the Jupyter Notebook file browser to navigate to the directory where you want to create a new notebook.

2. **Create New Notebook:** Click the New button on the right-hand side and select Python 3 (or your preferred environment) from the drop-down menu. This will open a new notebook with an empty code cell.

Using Jupyter Notebook

Notebook Interface

1. **Cells:** The main building blocks of a notebook, where you write code, text, or equations.
    - **Code Cells:** Execute Python code and display the output directly below the cell.
    - **Markdown Cells:** Write formatted text using Markdown syntax.



2. **Toolbar:** Contains buttons for common actions such as running cells, stopping execution, and saving the notebook.
3. **Menu Bar:** Provides access to various features and functions, such as creating new cells, running the entire notebook, and exporting the notebook.

Running Code

1. **Write Code:** Type your Python code into a code cell.
2. **Execute Code:** Press Shift + Enter to run the code in the current cell and move to the next cell, or click the Run button in the toolbar.

Writing Markdown

1. **Switch to Markdown Cell:** Click on a cell and change the cell type to Markdown from the drop-down menu in the toolbar or press Esc followed by M.
2. **Write Text:** Use Markdown syntax to format text, create headings, lists, links, images, and more.
3. **Render Markdown:** Press Shift + Enter to render the Markdown cell.

3.3 Python Variable and Identifiers

Identifiers are names given to variables, functions, classes, and other elements in Python. Understanding naming conventions is crucial for writing readable and maintainable code.

Rules for Naming Identifiers

- **First Character**: Must be an alphabet (uppercase or lowercase) or an underscore (_).
- **Subsequent Characters**: Can be alphabets, digits, or underscores.

- **No Special Characters**: Characters like !, @, #, $, %, and punctuation symbols are not allowed.
- **No Successive Underscores**: Avoid using multiple consecutive underscores.
- **Keywords**: Reserved words in Python cannot be used as identifiers.

Note: Python is case-sensitive, so Variable and variable are considered different identifiers.

False	await	else	import	pass
None	break	except	in	raise
True	class	finally	is	return
and	continue	for	lambda	try
as	def	from	nonlocal	while
assert	del	global	not	with
async	elif	if	or	yield

Lines and Indentation

Indentation in Python is used to define blocks of code. Unlike other languages that use curly braces {}, Python relies on consistent spacing to denote code blocks.

Importance of Indentation

- **Syntax Requirement**: Python requires indentation to define blocks of code.
- **Block Definition**: Indentation indicates the start and end of a block.
- **Error Prevention**: Incorrect indentation results in syntax errors.

Example:

In this example, statement1, statement2, and statement4 are part of the main block, while statement3 is part of a nested block.

```
if condition:
    statement1
    statement2
    if another_condition:
        statement3
    statement4
```

Input From the User

In Python, you can get input from the user using the input() function. This function reads a line of text from the user and returns it as a string.

```
user_input = input("Enter something: ")
print("You entered:", user_input)

```

- input("Enter something: ") prompts the user with the message "Enter something: " and waits for them to type something.

- The entered text is returned as a string and stored in the variable user_input.
- print("You entered:", user_input) displays the input text.

Converting Input Types: By default, the input() function returns the user input as a string. If you need a different type (e.g., an integer), you'll need to convert it:

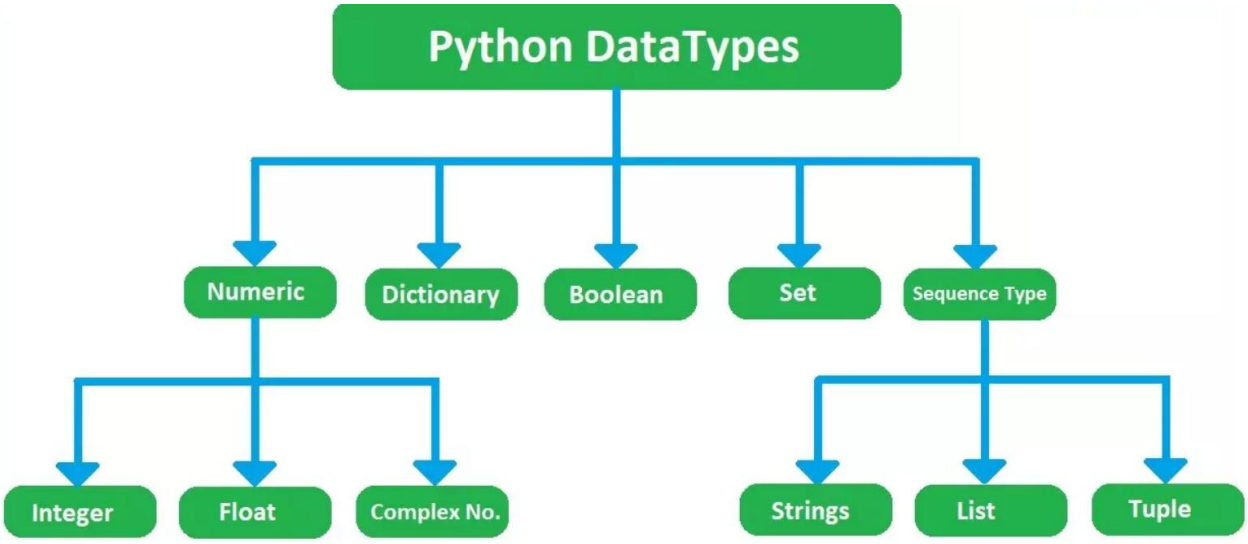
```
age = int(input("Enter your age: "))  # Converts the input to an integer
print("You are", age, "years old.")

```

If the user inputs something that cannot be converted to an integer, this will raise a ValueError.

Handling Input Errors: It's good practice to handle potential errors when converting input. For example:

3.4 Data Types and Operators



Source: https://www.theengineeringprojects.com/2020/06/how-to-use-data-types-in-python.html

Numeric Data

Numeric types represent numbers and include integers, floating-point numbers, and complex numbers.

int: Integer numbers, e.g., 42, -3, 0

```python
# Assigning integers to variables
a = 42
b = -3
c = 0
# Basic arithmetic operations
addition = a + b        # 42 + (-3) = 39
subtraction = a - b     # 42 - (-3) = 45
multiplication = a * b  # 42 * (-3) = -126
division = a / b        # 42 / (-3) = -14.0 (result is a float)
floor_division = a // b # 42 // (-3) = -14 (result is an integer)
modulus = a % b         # 42 % (-3) = 0
exponentiation = a ** 2 # 42 ** 2 = 1764
# Printing the results
print("Addition:", addition)
print("Subtraction:", subtraction)
print("Multiplication:", multiplication)
print("Division:", division)
print("Floor Division:", floor_division)
print("Modulus:", modulus)
print("Exponentiation:", exponentiation)
```

✓ 0.0s

```
Addition: 39
Subtraction: 45
Multiplication: -126
Division: -14.0
Floor Division: -14
Modulus: 0
Exponentiation: 1764
```

float: Floating-point numbers (decimal numbers), e.g., 3.14, -2.7

```python
# Assigning floating-point numbers to variables
a = 3.14
b = -2.7
c = 0.0

# Basic arithmetic operations
addition = a + b        # 3.14 + (-2.7) = 0.4400000000000004
subtraction = a - b     # 3.14 - (-2.7) = 5.84
multiplication = a * b  # 3.14 * (-2.7) = -8.478
division = a / b        # 3.14 / (-2.7) = -1.1629629629629628
floor_division = a // b # 3.14 // (-2.7) = -2.0 (result is a float)
modulus = a % b         # 3.14 % (-2.7) = -2.26
exponentiation = a ** 2 # 3.14 ** 2 = 9.8596

# Printing the results
print("Addition:", addition)
print("Subtraction:", subtraction)
print("Multiplication:", multiplication)
print("Division:", division)
print("Floor Division:", floor_division)
print("Modulus:", modulus)
print("Exponentiation:", exponentiation)
```

✓ 0.0s

```
Addition: 0.43999999999999995
Subtraction: 5.84
Multiplication: -8.478000000000002
Division: -1.162962962962963
Floor Division: -2.0
Modulus: -2.2600000000000002
Exponentiation: 9.8596
```

complex: Complex numbers, e.g., 1+2j, -3+4j

```python
# Assigning complex numbers to variables
a = 1 + 2j
b = -3 + 4j
# Basic arithmetic operations
multiplication = a * b    # (1+2j) * (-3+4j) = (-11-2j)
division = a / b          # (1+2j) / (-3+4j) = (0.2-0.4j)
# Conjugate of a complex number
conjugate_a = a.conjugate()  # Conjugate of (1+2j) = (1-2j)
# Magnitude (absolute value) of a complex number
magnitude_a = abs(a)      # Magnitude of (1+2j) = sqrt(1^2 + 2^2) = sqrt(5)
# Real and imaginary parts
real_part = a.real        # Real part of (1+2j) = 1.0
imaginary_part = a.imag   # Imaginary part of (1+2j) = 2.0
# Printing the results
print("Multiplication:", multiplication)
print("Division:", division)
print("Conjugate:", conjugate_a)
print("Magnitude:", magnitude_a)
print("Real Part:", real_part)
print("Imaginary Part:", imaginary_part)
```

✓ 0.0s

```
Multiplication: (-11-2j)
Division: (0.2-0.4j)
Conjugate: (1-2j)
Magnitude: 2.23606797749979
Real Part: 1.0
Imaginary Part: 2.0
```

Sequence Data Types

In Python, sequence data types are collections of elements that support indexing, slicing, and iteration. They include the following types:

- Lists
- Tuples
- Strings

List in Python

Python offers six types of built-in sequences to store data, one of which is a List. The List is like a dynamic array, along with the linear sequence, and also provides you with several built-in functions to perform various operations on data stored in it.

Python offers six types of built-in sequences to store data, one of which is a List. The List is like a dynamic array, along with the linear sequence, and also provides you with several built-in functions to perform various operations on data stored in it. A list can also be both homogenous as well as heterogeneous. It means we can store only integers or strings or both depending on the need. Next, every element rests at some position (i.e., index) in the list. The index can be used later to locate a particular item. The first indexes begin at zero, next are one, and so forth.

Characteristics of a Python List

The various characteristics of a list are:

- Ordered: Lists maintain the order in which the data is inserted.
- Mutable: In list element(s) are changeable. It means that we can modify the items stored within the list.
- Heterogenous: Lists can store elements of various data types.
- Dynamic: List can expand or shrink automatically to accommodate the items accordingly.
- Duplicate Elements: Lists allow us to store duplicate data.

Creating Lists in Python

A list is created by placing the items/ elements separated by a comma (,) between the square brackets ([ ]). Let's see how we can create Lists in Python in different ways.

```python
# Creation of a List in Python

# Creating an empty List
empty_List = []

# Creating a List of Integers
integer_List = [26, 12, 97, 8]

# Creating a List of floating point numbers
float_List = [5.8, 12.0, 9.7, 10.8]
# Creating a List of Strings
string_List = ["Edunet Foundation", "Preparation", "India"]

# Creating a List containing items of different data types
List = [1, "Edunet Foundation", 9.5, 'D']

# Creating a List containing duplicate items
duplicate_List = [1, 2, 1, 1, 3,3, 5, 8, 8]
```
✓ 0.0s

We can also create a list storing several other lists as its items. It is known as a nested list. The nested list is created by placing the various lists separated by comma (,) within the square brackets ([ ]).

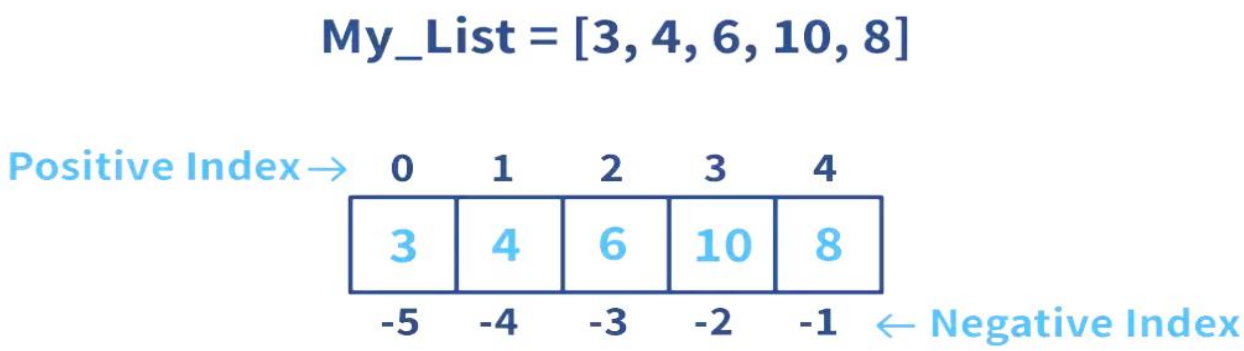
Let us see below how nested lists are created in Python.

```python
# Creating a nested List
nested_List = [[6, 2, 8], [1, "Edunet Foundation", 3.5], "preparation"]
```
✓ 0.0s

Accessing Values/Elements in List in Python

Elements stored in the lists are associated with a unique integer number known as an index. The first element is indexed as 0, and the second is 1, and so on. So, a list containing six elements will have an index from 0 to 5. For accessing the elements in the List, the index is mentioned within the index operator ([ ]) preceded by the list's name.

Another way we can access elements/values from the list in python is using a negative index. The negative index starts at -1 for the last element, -2 for the last second element, and so on.



Source: https://www.scaler.com/topics/python/

Accessing Values

```python
My_List = [3, 4, 6, 10, 8]

# Accessing values stored in lists using positive index
print("Values accessed using positive Index.")
print(My_List[2])
print(My_List[4])

# Accessing values stored in lists using negative index
print("Values accessed using negative Index.")
print(My_List[-1])
print(My_List[-5])
```

✓ 0.0s

```
Values accessed using positive Index.
6
8
Values accessed using negative Index.
8
3
```

We also have nested Lists in python. For nested Lists, we first access the index at which the inner list is stored/indexed in the outer list using the index operator. Then we access the element from the inner list using the index operator again. It is known as nested indexing. It is like we access elements in a 2-dimensional array.

```python
# Nested List
nested_List = [[6, 2, 8], [1, 3.5, "Edunet Foundation"], "preparation"]
print(nested_List[0][1])
print(nested_List[1][2])
```

✓ 0.0s

```
2
Edunet Foundation
```

Updating List in Python

Lists are mutable, meaning that their existing values stored at the indexes can be changed. To change or update the list elements, the assignment operator (=) is used.

```python
My_List = [3, 4, 6, 10, 8, 5]
print(My_List)
# Changing value to index 3.
My_List[3] = 7
print(My_List)
# Changing value to index -1, i.e the last value.
My_List[-1] = 15
print(My_List)
```

✓ 0.0s

```
[3, 4, 6, 10, 8, 5]
[3, 4, 6, 7, 8, 5]
[3, 4, 6, 7, 8, 15]
```

Add/Change list Elements in Python

Lists are mutable, unlike strings, so we can add elements to the lists. We use the append() method to add a single element at the end of the list and the extend() method to add multiple elements at the end of the list. We also have an insert() method to insert an element at the specified index position. The insert() method takes two arguments, the first is the index at which the element is to be inserted, and the second is the value/ element to be inserted.

```python
vowels = ['a', 'e']
print(vowels)
# Appending an element in list
vowels.append('i')
print(vowels)
# Extending an element in list
vowels.extend(['o', 'u'])
print(vowels)
# Inserting an element at a specified index
Integers = [1, 5, 6, 9]
Integers.insert(1,7)
print(Integers)
```

✓ 0.0s

```
['a', 'e']
['a', 'e', 'i']
['a', 'e', 'i', 'o', 'u']
[1, 7, 5, 6, 9]
```

Slicing a List in Python

In Python, we can access the elements stored in a range of indexes using the slicing operator (:). The index put on the left side of the slicing operator is inclusive, and that mentioned on the right side is excluded.

Slice itself means part of something. So when we want to access elements stored at some part or range in a list we use slicing.

```python
my_List = ['e', 'd', 'u', 'n', 'e','t', 'f', 'o', 'u', 'n','d','a','t','i','o','n']
# Accessing elements from index beginning to 5th index
print(my_List[:6])
# Accessing elements from index 3 to 6
print(my_List[3:7])
# Accessing elements from index 5 to end
print(my_List[5:])
# Accessing elements from beginning to end
print(my_List[:])
# Accessing elements from beginning to 4th index
print(my_List[:-6])

```

✓ 0.0s

```
['e', 'd', 'u', 'n', 'e', 't']
['n', 'e', 't', 'f']
['t', 'f', 'o', 'u', 'n', 'd', 'a', 't', 'i', 'o', 'n']
['e', 'd', 'u', 'n', 'e', 't', 'f', 'o', 'u', 'n', 'd', 'a', 't', 'i', 'o', 'n']
['e', 'd', 'u', 'n', 'e', 't', 'f', 'o', 'u', 'n']
```

We can also use slicing to change, remove/delete multiple elements from the list. Let's see some code examples and understand them.

```
List = [1, 2, 3, 4, 5, 6, 7, 8]
print(List)
# Changing elements from index 2nd to 5th
List[2:5] = [1, 1, 1]
print(List)
# Deletion multiple items from the List
del List[2:6]
print(List)
```

✓ 0.0s

```
[1, 2, 3, 4, 5, 6, 7, 8]
[1, 2, 1, 1, 1, 6, 7, 8]
[1, 2, 7, 8]
```

List Methods in Python

In python, the method is like a function that performs some task but can only be called on objects. Methods are related to objects and may change the object's state. Let's see some python list methods.

Method	Description of method
append()	Add an item to the ending of the list.
extend()	Add items of one list to another list.
insert()	Adds an item at the desired index position.
remove()	Remove the primary item from the list with the required value.
pop()	Remove the item from the required index within the list, and return it.
clear()	Remove all items from the list.
index()	Returns 0-based index of the first matched item. Raises ValueError if an item isn't found within the list.
count()	Returns the frequency of an element appearing in the list.
sort()	Sorts the objects of the list. It uses a compare function if passed as an argument.
reverse()	Reverse the order of the list items in place.
copy()	Return a shallow copy of the list i.e. two lists share the identical elements via reference.

Tuples in Python

Tuples in python is one of the four inbuilt data types used to store collections in Python. Unlike other data types, the elements in tuples are ordered and immutable. They are used to store multiple items in a single variable and provides some built-in operation to work with them.

Creating a Tuple in Python

Tuples in Python can only be created when they are being assigned, hence placing all the elements inside the parenthesis, separated by a comma will create a tuple.

Let's take a closer look at the syntax:

```python
tempTuple = ('apple', 'mango', 'banana')
print(tempTuple)
```

✓ 0.0s

```
('apple', 'mango', 'banana')
```

The parenthesis in the above syntax are optional and hence a tuple can also be created without writing the parenthesis. But remember it's always a good practice to have the parenthesis, as it always increases the code readability and understanding.

Immutable in Tuples

Tuples in Python are similar to lists but with a key difference: they are immutable. This means once a tuple is created, its elements cannot be changed, added, or removed. Let's break down some characteristics of tuples:

- Immutable Nature: Once a tuple is created, its elements cannot be modified. This ensures data integrity and stability.
- Ordered Collection: Like lists, tuples maintain the order of elements as they are inserted.
- Support for Duplicate Values: Tuples allow duplicate values just like lists.
- Accessing Elements: Elements in a tuple can be accessed using their index values. Here's an example illustrating the immutability of tuples in python:

```python
my_tuple = (1, 2, 3)

#Attempting to change an element in the tuple will result in an error
my_tuple[1] = 1  # This will raise a TypeError
```

⊗ 0.0s

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[41], line 4
      1 my_tuple = (1, 2, 3)
      3 #Attempting to change an element in the tuple will result in an error
----> 4 my_tuple[1] = 1

TypeError: 'tuple' object does not support item assignment
```

When attempting to modify an element within a tuple, Python will raise a TypeError because tuples in python do not support item assignment due to their immutable nature. This constraint ensures that once data is stored in a tuple, it remains unchanged throughout the program's execution.

Accessing Elements in a Python Tuple and Indexing

Accessing elements in a tuple is no different then accessing elements in a list. As python follows 0 based indexing hence a tuple with n elements will have indices from 0 through n-1. An index in a tuple is accessed with the index operator [ ].

For example: Let's consider a basic tuple:

```
tempTuple = ('hello', 1, 2, 3)
print(tempTuple[0]) # prints first element of the tuple
print(tempTuple[3]) # prints last element of the tuple
print(tempTuple[4]) # error
```

0.0s

```
hello
3

---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
Cell In[42], line 4
      2 print(tempTuple[0]) # prints first element of the tuple
      3 print(tempTuple[3]) # prints last element of the tuple
----> 4 print(tempTuple[4])

IndexError: tuple index out of range
```

Strings in Python

A string is just a sequence of characters. It is among the most popular data types in Python. It can be created simply by enclosing characters in quotes.

An immutable data sequence is known as a string in Python. In simple words, as discussed in the case of a crossword, a Python string is nothing but an array of characters, but a computer does not understand characters. It only understands the language of 0's and 1's. So these characters are converted to a number so the computer can understand them clearly. This conversion is known as encoding. ASCII and Unicode are some of the popular encodings used.

Creating a String in Python

A string in Python can be easily created using single, double, or even triple quotes. The characters that form the string are enclosed within any of these quotes.

Note: Triple quotes are generally used when we are working with multiline strings in Python and docstring in Python.

Let's take a look at an example where we will create strings using the three types of quotes,

```python
single_string='Hello'
double_string="Hello Eduent Team"
triple_string="""Welcome to Edunet Foundation"""
print(single_string)
print(double_string)
print(triple_string)
```

✓ 0.0s

```
Hello
Hello Eduent Team
Welcome to Edunet Foundation
```

Reassigning Strings in Python

Since strings in Python are immutable, we can update the values of strings simply by reassigning these strings with new values. The strings cannot be partially replaced, so it is always replaced completely with a new string. Let's see how that's done

```python
message="Welcome to Edunet Foundation"
print(message)
message="Hello World" #reassigning a new string
print(message)
```

✓ 0.0s

```
Welcome to Edunet Foundation
Hello World
```

Access Characters in a Python string

Indexing: The length of a string can be easily calculated using the len() function. The length of a string provides us with a range of the string. In python, we can perform indexing on any string as long as it is within the range. The index is always referred to as an integer. Using any other data type would give rise to a TypeError.

Indexing can either be positive or negative. Positive indexing starts from 0 and indexing from the beginning of the string. On the other hand, negative indexing starts from -1, and indexing starts from the end of the string.

```python
message="Welcome to Edunet Foundation"
len_str= len(message)
print(len_str)
print(message[4]) # positive indexing within the range
print(message[-2]) #negative indexing within the range
print(message[33]) #indexing outside the range

```

0.0s

```
28
o
o

---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
Cell In[62], line 6
      4 print(message[4]) # positive indexing within the range
      5 print(message[-2]) #negative indexing within the range
----> 6 print(message[33])

IndexError: string index out of range
```

Using the len() function, we calculated the length of the string variable message. We performed positive and negative indexing on this Python string. The thing to note is that we got an IndexError when the index value was way beyond the range of the string.

Slicing

Unlike indexing, slicing returns a range of characters using the slicing syntax. While performing slicing operations, one thing to remember is that the start index value is always included while that of the last index is always excluded.

```python
message="Welcome to Edunet Foundation"
print(message[2:5]) # positive slicing
print(message[-10:-2]) #negative slicing
print(message[:5]) #slicing from the start
print(message[2:]) #slicing to the end
```

0.0s

```
lco
Foundati
Welco
lcome to Edunet Foundation
```

Dictionary in Python

The data is stored in a key-value pair format using Python Dictionary. In Python, a dictionary is a data type that may replicate a real-world data arrangement in which a given value exists for a specified key. It's a data structure that can be changed. The keys and values define a Dictionary. Keys must be made up of only one element. Values can be of any type, including list, tuple, integer, etc. A dictionary, in other words, is a collection of key-value pairs with the value being any Python object. On the other hand, the keys are immutable Python objects such as Numbers, strings, or tuples.

Dictionary in Python is one of the most popular data structures. They are used to store data in a key-value pair format. The keys are always unique within a dictionary and are the attribute that helps us locate the data in the memory.

The values of the Python dictionary may or may not be unique. The values within a dictionary can be of any data type, but the thing to note is that the keys are immutable. Hence, the key can only be strings, numbers or tuples.

Creating a Dictionary in Python

Creating a Python dictionary is as simple as placing the required key and value pairs within a curly bracket.

“{}”.

A colon separates the key-value pair.

“:”

When there are multiple key-value pairs, they are separated by a comma.

“,”

```
#The syntax of declaring a dictionary in python is as follows

my_dict={'Name':'Ravi',"Age":'32'}
```

✓ 0.0s

creating a Python dictionary,

```python
#creating an empty dictionary
my_dict={}
#creating a dictionary with integer keys
fruits={'1':'apple','2':'banana','3':'cherry'}
#creating a dictionary with mixed keys
random_dict={'1':'red','Name':'Anushka'}
print(my_dict)
print(fruits)
print(random_dict)
```

✓ 0.0s

```
{}
{'1': 'apple', '2': 'banana', '3': 'cherry'}
{'1': 'red', 'Name': 'Anushka'}
```

Another way we can create a dictionary in Python is using the built-in dict() function!

Let's see how we can do that,

```python
Dict = dict([(1, 'Scaler'), (2, 'Academy')])
print("\nCreate a Dictionary by using  dict(): ")
print(Dict)
```

✓ 0.0s

```
Create a Dictionary by using  dict():
{1: 'Scaler', 2: 'Academy'}
```

Accessing values in Python Dictionary

We can use our favourite square brackets to access the values in a Python Dictionary! (Note- Square brackets are also commonly used for indexing and slicing strings in Python!)

```python
my_dict={'Name':'Ramar',"Age":'32','ID':'12345'}
print(my_dict['ID']) #accessing using the ID key
print(my_dict['Age']) #accessing using the Age
```

✓ 0.0s

```
12345
32
```

Updating Dictionary in Python

We can do multiple things when trying to update a dictionary in Python. Some of them are –

- Add a new entry.

- Modify an existing entry.
- Adding multiple values to a single key.
- Adding a nested key.

```python
#creating an empty dictionary
my_dict={}
print(my_dict)
#adding elements to the dictionary one at a time
my_dict[1]="Ramar"
my_dict[2]="Joy"
my_dict[3]="Rashmi"
print(my_dict)
#modifying existing entry
my_dict[2]="Bharti"
print(my_dict)
#adding multiple values to a single key
my_dict[4]="Soumya, Kishore, Kumar"
print(my_dict)
#adding nested key values
my_dict[5]={'Nested' :{'1' : 'Edunet', '2' : 'Foundation'}}
print(my_dict)
```

✓ 0.0s

```
{}
{1: 'Ramar', 2: 'Joy', 3: 'Rashmi'}
{1: 'Ramar', 2: 'Bharti', 3: 'Rashmi'}
{1: 'Ramar', 2: 'Bharti', 3: 'Rashmi', 4: 'Soumya, Kishore, Kumar'}
{1: 'Ramar', 2: 'Bharti', 3: 'Rashmi', 4: 'Soumya, Kishore, Kumar', 5: {'Nested': {'1': 'Edunet', '2': 'Foundation'}}}
```

Sets in Python

In mathematics, a set is a collection of distinct objects forming a group. The objects can be of any type like numbers, names of movies, company names, etc. There can be any number of objects in a set and the objects can be of different data types. Set is a concept of Set Theory which is a popular and useful topic of mathematics. There are several operations that can be performed on a set, these operations include union, intersection, difference.

Sets in python are also similar but with a few conditions.

- First, that a set cannot contain duplicate elements.
- Second, the elements of a set are immutable(once entered cannot be changed) but the set as a whole is mutable.

Creating Sets in Python

Just like most other things, creating a set in Python is really simple. There are two ways of creating a set

1. Putting all the elements inside curly braces “{“ and “}”
2. Using set() method.

If the curly braces look familiar to you, it's because they are used in Python Dictionaries.

Using Curly Braces, Let's look at an example,

```python
evenNumber = {2, 4, 6}
print(evenNumber)
stringAndNum = {"one", 2, 3}
print(stringAndNum)
```
✓ 0.0s

```
{2, 4, 6}
{'one', 2, 3}
```

Using set() function

Return type: Empty is no argument is given otherwise a set or frozenset object

set() takes an **iterable as an argument**. In python, string, lists, and dictionaries are iterable so you can pass them inside set(). set() returns a set or frozenset object, If there is no input (i.e. no iterable passed as argument) then an empty set is returned.

```python
setWithMethod = set([1, 2, 'three'])
print(setWithMethod)
```
✓ 0.0s

```
{1, 2, 'three'}
```

Here is a code example that shows how you can make a set from iterables (lists, dictionaries, string, etc.)

```python
a = {"one": 1, "two": 2}
b = [100, 200, 300]
c = 'python'
print(set(a), end='\n')
print(set(b), end='\n')
print(set(c))
```
✓ 0.0s

```
{'one', 'two'}
{200, 100, 300}
{'h', 't', 'p', 'n', 'y', 'o'}
```

Adding Items in a Python Set

Adding elements in a set is not the same as adding elements in lists. **Set do not use indexing.** There are two methods to add elements in a set

1. add() allows adding single element
2. update() allows to add multiple elements

Let's take a look at the code,

```python
initialSet = {1, 2}
initialSet.add(3)
print(initialSet)
toAdd = [4, 5]
initialSet.update(toAdd)
print(initialSet)
```

✓ 0.0s

```
{1, 2, 3}
{1, 2, 3, 4, 5}
```

Operators in Python

In Python, operators are special symbols that designate computations to be performed. The values that an operator acts on are called operands.

In this case, the + operator adds the operands a and b together. An operand can be either a literal value or a variable that references an object:

A sequence of operands and operators, like a + b - 5, is called an expression. Python supports many operators for combining data objects into expressions, which are explored below.

Types of Operators

Python supports several types of operators, each serving different purposes. These operators are classified into the following categories:

- Python Arithmetic Operators
- Python Comparison Operators
- Python Assignment Operators
- Python Bitwise Operators
- Python Logical Operators
- Python Membership Operators
- Python Identity Operators

These operators form the basis for performing various operations in Python programming.

Python Arithmetic Operators

Python provides several operators to perform basic arithmetic operations. These operators are used to perform mathematical operations such as addition, subtraction, multiplication, and division. Below is a list of the most commonly used arithmetic operators in Python:

Addition (+)

The addition operator adds two operands.

```python
num1 = 15
num2 = 10
sum = num1 + num2
print(f"The sum of {num1} and {num2} is {sum}")
```

✓ 0.0s

```
The sum of 15 and 10 is 25
```

Subtraction (-)

The subtraction operator subtracts the second operand from the first.

```python
num1 = 15
num2 = 10
difference = num1 - num2
print(f"The difference between {num1} and {num2} is {difference}")
```

✓ 0.0s

```
The difference between 15 and 10 is 5
```

Multiplication (*)

The multiplication operator multiplies two operands.

```python
num1 = 15
num2 = 10
product = num1 * num2
print(f"The product of {num1} and {num2} is {product}")
```

✓ 0.0s

```
The product of 15 and 10 is 150
```

Division (/)

The division operator divides the first operand by the second. It always returns a float value.

```python
num1 = 15
num2 = 10
quotient = num1 / num2
print(f"The quotient when {num1} is divided by {num2} is {quotient}")
```

✓ 0.0s

```
The quotient when 15 is divided by 10 is 1.5
```

Floor Division (//)

The floor division operator divides the first operand by the second and returns the largest possible integer (floored value).

```python
num1 = 15
num2 = 10
floor_div = num1 // num2
print(f"The floor division of {num1} by {num2} is {floor_div}")
```

✓ 0.0s

```
The floor division of 15 by 10 is 1
```

Modulus (%)

The modulus operator returns the remainder of the division of the first operand by the second.

```python
num1 = 15
num2 = 10
remainder = num1 % num2
print(f"The remainder when {num1} is divided by {num2} is {remainder}")
```

✓ 0.0s

```
The remainder when 15 is divided by 10 is 5
```

Exponentiation (**)

The exponentiation operator raises the first operand to the power of the second operand.

```python
base = 2
exponent = 3
power = base ** exponent
print(f"{base} to the power of {exponent} is {power}")
```

✓ 0.0s

```
2 to the power of 3 is 8
```

Python Comparison Operators

Comparison operators in Python are used to compare values. They return a Boolean value (True or False) based on the comparison. These operators are essential for making decisions and controlling the flow of execution in your code.

Equal to (==)

The == operator checks if the values of two operands are equal.

```python
x = 5
y = 5
print(x == y)
```

✓ 0.0s

```
True
```

Not equal to (!=)

The != operator checks if the values of two operands are not equal.

```
a = 10
b = 20
result = (a != b)
print(result)
```

✓ 0.0s

```
True
```

Greater than (>)

The > operator checks if the value of the left operand is greater than the value of the right operand.

```
a = 10
b = 20
result = (a > b)
print(result)
```

✓ 0.0s

```
False
```

Less than (<)

The < operator checks if the value of the left operand is less than the value of the right operand.

```
a = 10
b = 20
result = (a < b)
print(result)
```

✓ 0.0s

```
True
```

Greater than or equal to (>=)

The >= operator checks if the value of the left operand is greater than or equal to the value of the right operand.

```
a = 10
b = 20
result = (a >= b)
print(result)
```

✓ 0.0s

```
False
```

Less than or equal to (<=)

The <= operator checks if the value of the left operand is less than or equal to the value of the right operand.

```
a = 10
b = 20
result = (a <= b)
print(result)
```

✓ 0.0s

```
True
```

An overview of comparison operators in Python, along with examples to help you understand their usage.

Python Assignment Operators

Assignment operators in Python are used to assign values to variables. In addition to the simple assignment operator, Python provides a set of shorthand assignment operators to perform an operation and assign the result to the left operand in a single step.

Assign (=)

The = operator assigns the value of the right operand to the left operand.

```
a = 10
print(a)
```

✓ 0.0s

```
10
```

Add and Assign (+=)

The += operator adds the right operand to the left operand and assigns the result to the left operand.

```
a = 10
a += 5
print(a)
```

✓ 0.0s

```
15
```

Subtract and Assign (-=)

The -= operator subtracts the right operand from the left operand and assigns the result to the left operand.

```
a = 10
a -= 5
print(a)
```

✓ 0.0s

```
5
```

Multiply and Assign (*=)

The *= operator multiplies the left operand by the right operand and assigns the result to the left operand.

```
a = 10
a *= 5
print(a)
```

✓ 0.0s

```
50
```

Divide and Assign (/=)

The /= operator divides the left operand by the right operand and assigns the result to the left operand. The result is a float.

```
a = 10
a /= 5
print(a)
```

✓ 0.0s

```
2.0
```

Modulus and Assign (%=)

The %= operator takes the modulus using the two operands and assigns the result to the left operand.

```
a = 10
a %= 3
print(a)
```

✓ 0.0s

```
1
```

Floor Divide and Assign (//=)

The //= operator performs floor division on the two operands and assigns the result to the left operand.

```
a = 10
a //= 3
print(a)
```

✓ 0.0s

```
3
```

Exponent and Assign (=)**

The **= operator raises the left operand to the power of the right operand and assigns the result to the left operand.

```
a = 2
a **= 3
print(a)
```

✓ 0.0s

```
8
```

Python Bitwise Operators

Bitwise operators in Python are used to perform operations at the binary level. They operate on binary digits or bits and perform bit-by-bit operations. These operators are particularly useful for low-level programming, such as working with hardware and performing efficient calculations.

Bitwise AND (&)

The bitwise AND operator compares each bit of the first operand to the corresponding bit of the second operand. If both bits are 1, the corresponding result bit is set to 1. Otherwise, the result bit is set to 0.

```
x = 12  # binary: 1100
y = 5   # binary: 0101
result = x & y
print(result)
```

✓ 0.0s

```
4
```

Bitwise OR (|)

The bitwise OR operator compares each bit of the first operand to the corresponding bit of the second operand. If either bit is 1, the corresponding result bit is set to 1. If both bits are 0, the result bit is set to 0.

```python
x = 12  # binary: 1100
y = 5   # binary: 0101
result = x ^ y
print(result)  # Output: 9 (binary: 1001)
```

✓ 0.0s

```
9
```

Bitwise XOR (^)

The bitwise XOR operator compares each bit of the first operand to the corresponding bit of the second operand. If the bits are different, the corresponding result bit is set to 1. If they are the same, the result bit is set to 0.

```python
x = 12  # binary: 1100
y = 5   # binary: 0101
result = x ^ y
print(result)  # Output: 9 (binary: 1001)
```

✓ 0.0s

```
9
```

Bitwise NOT (~)

The bitwise NOT operator inverts all the bits of the operand, turning 1s into 0s and 0s into 1s. This is also known as the bitwise complement.

```python
a = 10  # binary: 1010
result = ~a
print(result)  # Output: -11 (binary: -1011)
```

✓ 0.0s

```
-11
```

Bitwise Left Shift (<<)

The bitwise left shift operator shifts the bits of the first operand to the left by the number of positions specified by the second operand. Bits shifted off the end are discarded, and zeros are shifted in from the right.

```python
x = 3   # binary: 0011
result = x << 2
print(result)  # Output: 12 (binary: 1100)
```

✓ 0.0s

```
12
```

Bitwise Right Shift (>>)

The bitwise right shift operator shifts the bits of the first operand to the right by the number of positions specified by the second operand. Bits shifted off the end are discarded, and zeros are shifted in from the left for positive numbers.

```
a = 10  # binary: 1010
result = a >> 2
print(result)  # Output: 2 (binary: 0010)
```

✓ 0.0s

```
2
```

Python Logical Operators

Logical operators in Python are used to combine conditional statements. They return Boolean values (True or False) based on the logical relationship between the values or expressions. Python has three logical operators: and, or, and not.

Logical AND (and)

The and operator returns True if both operands are true. If either operand is false, it returns False.

```
x = 5
y = 10
result = (x > 2) and (y < 15)
print(result)
```

✓ 0.0s

```
True
```

Truth Table for Logical AND (and)

Operand 1	Operand 2	Result
TRUE	TRUE	TRUE
TRUE	FALSE	FALSE
FALSE	TRUE	FALSE
FALSE	FALSE	FALSE

Logical OR (or)

The or operator returns True if at least one of the operands is true. If both operands are false, it returns False.

```python
x = 5
y = 10
result = (x > 10) or (y < 15)
print(result)
```

✓ 0.0s

```
True
```

Truth Table for Logical OR (or)

Operand 1	Operand 2	Result
TRUE	TRUE	TRUE
TRUE	FALSE	TRUE
FALSE	TRUE	TRUE
FALSE	FALSE	FALSE

Logical NOT (not)

The not operator reverses the result of the operand; if the operand is true, it returns False, and if the operand is false, it returns True.

```python
x = 5
result = not (x > 2)
print(result)
```

✓ 0.0s

```
False
```

Truth Table for Logical NOT (not)

Operand	Result
TRUE	FALSE
FALSE	TRUE

Python Membership Operators

Membership operators in Python are used to test if a value or variable is found in a sequence (such as a string, list, tuple, set, or dictionary). There are two membership operators: in and not in.

in Operator

The in operator returns True if the specified value is present in the sequence, otherwise, it returns False.

```
a = [1, 2, 3, 4, 5]
result = 3 in a
print(result)
```
✓ 0.0s

```
True
```

not in Operator

The not in operator returns True if the specified value is not present in the sequence, otherwise, it returns False.

```
a = [1, 2, 3, 4, 5]
result = 6 not in a
print(result)
```
✓ 0.0s

```
True
```

Python Identity Operators

Identity operators in Python are used to compare the memory locations of two objects. There are two identity operators: is and is not.

is Operator

The is operator checks whether two variables refer to the same object in memory. It returns True if they do, and False otherwise.

```
a = 5
b = 5
result = (a is b)
print(result)
```
✓ 0.0s

```
True
```

is not Operator

The is not operator checks whether two variables do not refer to the same object in memory. It returns True if they do not, and False otherwise.

```
a = 5
b = 6
result = (a is not b)
print(result)
```
✓ 0.0s

```
True
```

3.5 Control Structures

3.5.1 Conditional Statements

Decision-making (conditional) statements are essential to programming because they enable programs to execute discrete code blocks in accordance with preset circumstances. Among the Python decision-making statements are the if, elif, and if-else statements. By employing these statements, programmers can specify how a program runs based on a range of criteria.

Types of conditional statements in python

There are four types of decision-making statements in Python language, those are

1. If statements in python
2. If else statement in python
3. Nested if else statement in python
4. if-elif-else Ladder in python

If statements in python

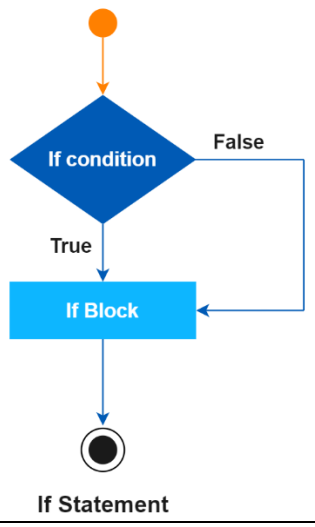
The if statement in Python is used for decision-making. It allows you to execute a block of code only if a certain condition is met.

Syntax of if Statements

The basic syntax of an if statement is:

```
if condition:
    # Block of code to be executed if the condition is true
```

Flow Chart



- The simplest structure for making decisions is an if statement, which only permits the execution of a code block if a predetermined condition is met.
- The if keyword is followed by an indented block of statements that are executed if the condition is assessed and found to be true.
- If not, the code skips the if block.

Source: https://www.scholarhat.com/tutorial/python/decision-making-statements-if-else-nested-if-else

Example of If Statement

```
age = 20
if age >= 18:
    print("You are eligible to vote.")
```

✓ 0.0s

```
You are eligible to vote.
```

If else statement in python

The if-else statement in Python is used to execute one block of code if a condition is true, and another block of code if the condition is false. This allows you to handle different outcomes based on the evaluation of a condition.

Syntax of if-else Statements

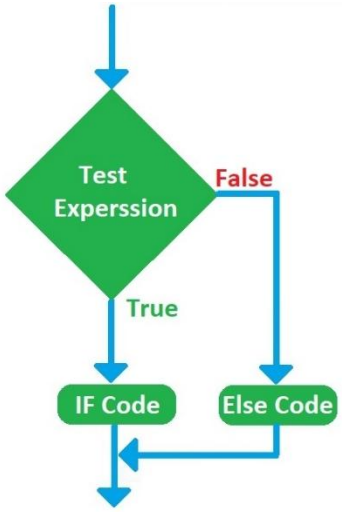
if condition:

    # Code to execute if the condition is true

else:

    # Code to execute if the condition is false

Flow Chart

	• Expression or condition: An expression that evaluates to True or False. • The block of code within the if statement is executed if the condition is true. • The block of code within the else statement is executed if the condition is false.

Source: https://www.theengineeringprojects.com/2020/06/how-to-use-if-else-in-python.html

Example of an if-else Statement

```
age = 16
if age >= 18:
    print("You are eligible to vote.")
else:
    print("You are not eligible to vote.")
```

✓ 0.0s

```
You are not eligible to vote.
```

3.5.2 Looping Statements

Looping statements in Python are used to repeatedly execute a block of code as long as a condition is met. They are fundamental for iterating over data, automating repetitive tasks, and controlling program flow.

Python provides two main looping constructs: for and while loops. The for loop iterates over sequences like lists, tuples, strings, or ranges, executing a block of code for each item. The while loop repeatedly executes a block of code as long as a specified condition remains True.

Types of Looping Statements

- for loop
- while loop

for Loop in Python

The for loop in Python is used to iterate over a sequence of elements, such as lists, tuples, strings, or ranges. It allows you to execute a block of code for each item in the sequence.

Syntax of for loop

for variable in sequence:

Code to execute for each item in the sequence

- **variable**: The name that will be assigned to the current item in the sequence.
- **sequence**: The iterable object you want to loop through, such as a list, tuple, string, or range.

Examples of for loop

Iterating Over a List

```python
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(fruit)
```

✓ 0.0s

```
apple
banana
cherry
```

Iterating Over a String

```python
word = "hello"
for letter in word:
    print(letter)

```

✓ 0.0s

```
h
e
l
l
o
```

Using range() Function

The range() function generates a sequence of numbers, which can be used to iterate a specific number of times.

```python
for i in range(5):
    print(i)
```

✓ 0.0s

```
0
1
2
3
4
```

Nested for Loops

You can use nested for loops to iterate over multi-dimensional sequences.

```python
matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]
for row in matrix:
    for item in row:
        print(item, end=' ')
    print()  # For newline after each row
```

✓ 0.0s

```
1 2 3
4 5 6
7 8 9
```

while Loop in Python

The while loop in Python is used to repeatedly execute a block of code as long as a specified condition remains True. It is useful for scenarios where the number of iterations is not known in advance and depends on dynamic conditions.

Syntax of while loop

while condition:

    # Code to execute as long as the condition is true

The **condition**: An expression that is evaluated before each iteration. If it evaluates to True, the code block inside the while loop executes. If it evaluates to False, the loop terminates.

Simple While Loop

```python
count = 1
while count <= 5:
    print(count)
    count += 1  # Increment count to eventually end the loop
```

✓ 0.0s

```
1
2
3
4
5
```

Using While with a Condition

This example demonstrates using while to validate user input until it's correct.

```python
password = ""
while password != "securepassword":
    password = input("Enter the password: ")
print("Access granted.")
```

✓ 35.0s

```
Access granted.
```

3.6 Functions in Python

In Python, a function is a block of reusable code that performs a specific task. Functions help to make code more modular, readable, and maintainable. By defining a function, you can encapsulate a set of instructions and execute them whenever needed without rewriting the same code.

3.6.1 Defining a Function

To define a function in Python, you use the **`def`** keyword followed by the function name, parentheses, and a colon. The code block within the function is indented.

```
def function_name(parameters):
    # Code block
    statement
```

Example:

```python
def greet(name):
    print(f"Hello, {name}!")

```

In this example, `greet` is the function name, and `name` is a parameter.

3.6.2 Calling a Function

Once a function is defined, you can call it by using its name followed by parentheses. If the function requires arguments, you need to pass them within the parentheses.

Example:

```python
def greet(name):
    print(f"Hello, {name}!")

greet("Alice")
```

```
Hello, Alice!
```

Parameters and Arguments

In programming, **parameters** and **arguments** are fundamental concepts used in the definition and invocation of functions. Understanding the difference between them is crucial for effective function usage and programming.

Parameters

Parameters are variables specified in the function definition. They serve as placeholders that define what kind of data the function expects to receive. When you define a function, list these parameters inside the parentheses following the function's name.

Example:

```
def greet(name, age):
    print(f"Hello, {name}! You are {age} years old.")
```

In this function definition, `name` and `age` are parameters. They act as placeholders for the actual values that will be provided when the function is called. Parameters define what inputs the function can accept and how it will process them.

Arguments

Arguments are the actual values or data you provide to a function when you call it. They replace the parameters in the function definition and are used to perform operations defined by the function.

Example:

```
greet("Alice", 30)
```

In this function call, `"Alice"` and `30` are arguments. They correspond to the `name` and `age` parameters, respectively. When the `greet` function is invoked with these arguments, it will replace the parameters with these values and execute the function's code, resulting in the output: `Hello, Alice! You are 30 years old.`

```
def greet(name, age):
    print(f"Hello, {name}! You are {age} years old.")

greet("Alice", 30)
```

✓ 0.0s

```
Hello, Alice! You are 30 years old.
```

Default Parameters

Default parameters in programming allow you to specify default values for function parameters. This feature is particularly useful when you want to provide default behaviour for a function if no specific value is supplied during the function call. Default parameters make

functions more flexible and can simplify code by reducing the need for multiple function overloads or additional logic to handle missing arguments.

Example:

```
def greet(name="World"):
    print(f"Hello, {name}!")

greet()  # Outputs: Hello, World!
greet("Alice")  # Outputs: Hello, Alice!
```

```
Hello, World!
Hello, Alice!
```

The `greet` function in Python showcases the use of default parameters and their behavior when called with or without arguments. In its definition, the function is specified as `def greet(name="World"):`, where `name` is a parameter with a default value of `"World"`. This means that if no value is provided for `name` when the function is called, it will automatically use `"World"` as the default.

When calling `greet()` without any arguments, the function utilizes this default value, resulting in the output: `Hello, World!`. This illustrates how default parameters provide a fallback value, simplifying function calls when specific values are not necessary.

In contrast, when `greet("Alice")` is called, the argument `"Alice"` overrides the default parameter. Consequently, the output is `Hello, Alice!`

Keyword Arguments

Keyword arguments in programming allow you to specify arguments by name when calling a function. This approach enhances code clarity and flexibility by explicitly identifying which parameter each argument corresponds to, rather than relying solely on the order of arguments.

What Are Keyword Arguments?

When you use keyword arguments, you provide the name of the parameter followed by an equal sign and the value you want to pass. This method allows you to bypass the order of parameters and makes your function calls more readable, especially when dealing with functions that have many parameters or default values.

Example:

```
def describe_pet(animal_type, pet_name):
    print(f"I have a {animal_type} named {pet_name}.")

describe_pet(animal_type="hamster", pet_name="Harry")
```

The describe_pet function exemplifies the use of keyword arguments to enhance clarity and precision in function calls. Defined as def describe_pet(animal_type, pet_name):, this function takes two parameters: animal_type and pet_name. These parameters are placeholders for the type of animal and the pet's name, respectively. When the function is called with describe_pet(animal_type="hamster", pet_name="Harry"), keyword arguments are used to

specify the values directly. This means animal_type is set to "hamster" and pet_name is set to "Harry". As a result, the function outputs: I have a hamster named Harry.

```
I have a hamster named Harry.
```

Return Statement

Functions can return values using the `return` statement. If no `return` statement is provided, the function returns `None` by default.

Example:

```python
def square(number):
    return number * number

result = square(4)
print(result)
```

✓ 0.0s

```
16
```

The square function is designed to compute the square of a given number. Defined as def square(number):, this function takes one parameter, number, and returns its square by multiplying the number by itself. For example, when the function is called with the argument 4, as shown in result = square(4), it calculates 4×44 \times 44×4 which equals 16. The result of this calculation is stored in the variable result. The subsequent print(result) statement then outputs the value of result, which is 16.

3.7 Data Manipulation and Analysis: Working with Data

Manipulating and analysing data forms the core of data science. These skills are essential for bringing insights, building predictive models, and visualizing trends out of data. Manipulating data generally involves various operations such as cleaning the data, transforming it to other forms, aggregating the data, and possibly visualization. These operations are vital in understanding patterns or spotting anomalies and preparing your data for more complex analysis or machine learning workflows.

Python is a versatile and powerful language with lots of libraries that make manipulating and analyzing data smooth sailing. In the following chapters, we briefly introduce a few of the most fundamental libraries used when working with data.

Commonly Used Libraries

Pandas:

- Pandas is primarily used to manipulate and analyze data.
- Key Features: It contains data structures, DataFrames and Series which simplify the manipulation of structured data. It has capabilities in powerful data loading, cleaning, merging, and filtering.
- Usual applications include data cleaning, exploration, and transformation.

NumPy

- Used to support large, multi-dimensional arrays and matrices for various numerical operations
- The key feature of NumPy is efficiency in mathematical computation; it comes along with linear algebra, statistical operation functions, and random sampling.
- Common Use Cases: Array manipulation, mathematical computation, and basis for other data analysis libraries.

Matplotlib

- Use Case: For static, animated, and interactive visualizations
- Key Features: Matplotlib is a very flexible library. It gives control over the plot and charts, figures, colors, labels, and styles.
- Common Use Cases: Creation of line graphs, bar charts, histograms, and other basic visualizations.

Seaborn

- Seaborn is designed on top of Matplotlib and is especially tailored for statistical data visualization.
- Key Features: It takes the complexity out of some of the visuals that involve large datasets and statistical analyses, such as heatmaps, pair plots, and distribution plots.
- Common Use Cases: Plotting statistical relationships, distributions, and comparisons inside the data.

3.8 Pandas Library

Pandas is an open-source library, designed especially to manipulate and analyze data, and widely used within programs of Python. It is one of the best tools which makes it easier for data scientists and analysts to work with large data sets and also brings vast functionality in the field of data handling and cleaning with analysis.

Source: https://www.scaler.com/topics/architecture-of-data-warehouse/

Key Data Structures

Pandas uses two main data structures, and they enable working with data in all formats:

Series:

A Series is a one-dimensional labeled array capable of holding any data type, including integers, strings, floats, and much more. It can be thought of as a single column in a spreadsheet or a database table.

DataFrame:

A DataFrame is a two-dimensional, size-mutable, and heterogeneous tabular data structure. It can be considered as a complete spreadsheet or a SQL table with labeled axes (rows and columns). DataFrames can hold different data types across columns, making them versatile for various types of data analysis.

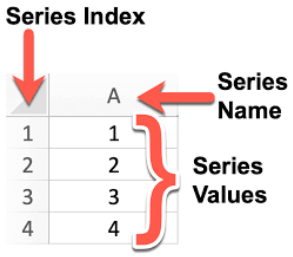
Key Features of Pandas

Pandas has several prominent features that make data manipulation and analysis easy.

- **High-Level Data Manipulation Tools:**
  Pandas provides extensive libraries and tools for reshaping, sorting, filtering, and aggregation of data. Such features make challenging tasks of data analysis easy and let the user dedicate their time to getting insights.
- **Data Cleaning, Transformation, and Aggregation:**
  Missing data can be handled easily with Pandas; calculations can be made on the data; formats can be changed; and aggregation can be done on data. This is necessary for preparation of datasets before any kind of analysis, to make sure that the data is relevant and correct.
- **Merging and Joining Datasets**:
  Pandas allows data from multiple sources to be merged and joined easily. In real-life analysis scenarios, data could originate from different files, databases, or APIs, making this feature necessary.

3.8.1 Pandas Series

Pandas Series is a labelled one-dimensional array of almost any data type (in other words, it may contain integers, strings, floats or even complex Python objects). It is precisely the same thing as a single column of an Excel sheet. You can easily organize and address data because values come attached with labels or indexes to every entry.



Source: https://codechalleng.es/bites/251/

Important Points about Pandas Series

- Flexible Data Types:
  - It means that a Series can accommodate many different types of data in one column, thus being flexible for many different types of data analysis and manipulation. For example, you can have integers, floats, strings, and even custom objects all within the same Series.

```python
import pandas as pd

# Creating a Series with different data types
mixed_data = pd.Series([1, 'two', 3.0, [4, 5], {'key': 'value'}])
print(mixed_data)

```

✓ 2.7s

```
0                   1
1                 two
2                 3.0
3              [4, 5]
4    {'key': 'value'}
dtype: object
```

- Intuitive Labelling
  - Every Series has an index label. Indexing allows for fast access, manipulation, and retrieval of specific elements in the Series. The index can be default numerical indices or custom labels assigned by the user.

```python
# Creating a Series with custom index labels
fruit_prices = pd.Series([1.5, 2.0, 3.0], index=['Apple', 'Banana', 'Cherry'])
print(fruit_prices)

```

✓ 0.0s

```
Apple     1.5
Banana    2.0
Cherry    3.0
dtype: float64
```

Creating a Pandas Series with the Series() Method

We can easily convert other data structures, such as lists, tuples, or dictionaries, into a Series object with the Series() method:

1. From a List or Tuple:
   - If you build a Series from a list or tuple, the elements are the values in the Series and will also by default acquire numeric indexing.

```python
# Creating a Series from a list
number_series = pd.Series([10, 20, 30, 40])
print(number_series)

```

✓ 0.0s

```
0    10
1    20
2    30
3    40
dtype: int64
```

2. From a Dictionary:
   - If you are building a Series from a dictionary, then the keys of the dictionary become index labels of the Series, while the values become the entries in the Series.

```
# Creating a Series from a dictionary
sales_data = {'Apple': 100, 'Banana': 150, 'Cherry': 200}
sales_series = pd.Series(sales_data)
print(sales_series)

✓ 0.0s
```

```
Apple     100
Banana    150
Cherry    200
dtype: int64
```

Accessing and Modifying Series:

- Accessing Values:
  - You can obtain values out of a Series via their index labels or integer locations.

```
# Accessing values by label
print(fruit_prices['Banana'])  # Output: 2.0

# Accessing values by position
print(fruit_prices[0])  # Output: 1.5

✓ 0.0s
```

```
2.0
1.5
```

- Updating Values:
  - Values in a Series may be updated quite easily. You could assign new values to any desired indices for updating.

```
# Modifying a value
fruit_prices['Cherry'] = 3.5
print(fruit_prices)

✓ 0.0s
```

```
Apple     1.5
Banana    2.0
Cherry    3.5
dtype: float64
```

3.8.2 Panda Data Frame

A Pandas DataFrame is the central data structure within the Pandas library and one of the most versatile tools that are found in data handling with Python. It's a two-dimensional data structure with labeled rows and columns, so it's very much like a table in any SQL database, an Excel spreadsheet, or a Google Sheet.

Key Characteristics of a DataFrame

- Two-Dimensional Format:
  - DataFrames are structured with rows and columns. They provide a structured method for users to store and manipulate large complex datasets. A row is an observation or record, and each column comprises a variable or feature. One can easily access certain points of data from DataFrames.
- Ease of intuitive resemblance with general data format:
  - The frame closely resembles ordinary data formats, such as a spreadsheet or SQL table, and understanding this becomes easy for users accustomed to those tools. Familiarity with this makes it easy to learn and use Pandas to manipulate and analyse data.

Three Major Parts of a DataFrame

1. **Data:**

- The actual data contained within the DataFrame, that can be more than one kind of data type—string, integer, float and even more complex data types like a list or dictionary.

```python
import pandas as pd

# Creating a DataFrame with different data types
data = {
    'Fruit': ['Apple', 'Banana', 'Cherry'],
    'Price': [1.5, 2.0, 3.0],
    'Quantity': [10, 5, 15],
    'Available': [True, False, True]
}
df = pd.DataFrame(data)
print(df)

```

✓ 3.2s

```
    Fruit  Price  Quantity  Available
0   Apple    1.5        10       True
1  Banana    2.0         5      False
2  Cherry    3.0        15       True
```

2. **Index:**

- Actual labels assigned to rows and help in uniquely identifying that row. Indexing allows flexibility for quick retrieval of data access. Pandas assigns default integer-based indexing but the user has options to change these indexes

```python
# Customizing the index
df.index = ['A', 'B', 'C']
print(df)

```

✓ 0.0s

```
    Fruit  Price  Quantity  Available
A   Apple    1.5        10       True
B  Banana    2.0         5      False
C  Cherry    3.0        15       True
```

3. **Columns:**

- Each column has a unique label permitting easy reference and filtering as well as sorting by the actual data points. You can access, modify or delete columns in a DataFrame using these labels.

```python
# Accessing a specific column
print(df['Price'])  # Output the 'Price' column

```

✓ 0.0s

```
A    1.5
B    2.0
C    3.0
Name: Price, dtype: float64
```

Creating and Working with DataFrames

DataFrames can be created from lists of lists, dictionaries of lists, external csv files, SQL databases, or even directly from other DataFrames. Such flexibility makes DataFrames great for exploratory data analysis, data manipulation, and exploration.

From a List of Lists:

You can read a DataFrame directly from a list of lists and provide column names.

```python
# Creating a DataFrame from a list of lists
data = [['Apple', 1.5, 10], ['Banana', 2.0, 5], ['Cherry', 3.0, 15]]
df_from_list = pd.DataFrame(data, columns=['Fruit', 'Price', 'Quantity'])
print(df_from_list)

```

✓ 0.0s

```
    Fruit  Price  Quantity
0   Apple    1.5        10
1  Banana    2.0         5
2  Cherry    3.0        15
```

3.8.3 Basic Operations with Pandas

- Pandas is a powerful data manipulation library in Python that provides a wide range of functions to work with structured data. Below are some basic operations you can perform using Pandas, including loading data, inspecting it, and selecting specific data subsets.

1. Loading Data

Pandas allows you to load data from various formats, making it easy to analyze and manipulate datasets. Here are some common methods:

- **Read CSV Files**:

```
import pandas as pd

# Load a CSV file into a DataFrame

df = pd.read_csv('data.csv')
```

- **Read Excel Files**:

```
# Load an Excel file into a DataFrame

df = pd.read_excel('data.xlsx', sheet_name='Sheet1') # Specify sheet name if needed
```

- **Read SQL Databases**:

```
from sqlalchemy import create_engine

# Create a database connection

engine = create_engine('sqlite:///database.db') # Example with SQLite

# Load data from a SQL query into a DataFrame

df = pd.read_sql_query('SELECT * FROM table_name', engine)
```

2. Data Inspection

Once you have loaded your data, it's essential to inspect it to understand its structure and contents.

- **View the First Few Rows**:

```
# Display the first 5 rows of the DataFrame

print(df.head())
```

- **Summary Statistics**:

```
# Get summary statistics for numerical columns

df.describe()
```

- **View DataFrame Information**:

```
# Display information about the DataFrame, including data types and non-null counts

print(df.info())
```

3. Data Selection

Pandas makes it easy to select specific rows and columns from a DataFrame.

- **Selecting Columns**:

  ```
  # Select a single column
  column_data = df['column_name']
  # Select multiple columns
  subset_df = df[['column1', 'column2']]
  ```

- **Selecting Rows**:

  ```
  # Select rows by index (e.g., first 10 rows)
  first_ten_rows = df.iloc[:10]
  # Select specific rows based on a condition
  filtered_df = df[df['column_name'] > value] # Replace 'value' with your condition
  ```

- **Using .loc[] for Label-based Selection**:

  ```
  # Select rows and columns by labels
  selected_data = df.loc[5:10, ['column1', 'column2']] # Rows 5 to 10 for specific columns
  ```

- **Using .iloc[] for Position-based Selection**:

  ```
  # Select rows and columns by index positions
  selected_data = df.iloc[0:5, 0:2] # First 5 rows and first 2 columns
  ```

Data Manipulation Techniques in Pandas

- Pandas provides a powerful set of tools for data manipulation, allowing users to filter, add or modify columns, and group data for aggregation. Here's a closer look at these techniques:

1. Filtering Data

Filtering is a crucial technique used to select subsets of data based on specific conditions.

- **Conditional Selection**: You can filter DataFrames using boolean indexing to retrieve rows that meet certain criteria.

  ```
  import pandas as pd
  # Example DataFrame
  data = { 'Name': ['Alice', 'Bob', 'Charlie', 'David'], 'Age': [24, 30, 22, 35], 'Salary': [50000, 60000, 52000, 58000] }
  df = pd.DataFrame(data)
  # Filter rows where Age is greater than 25
  filtered_df = df[df['Age'] > 25]
  ```

```
print(filtered_df)
```

- **Multiple Conditions**: You can also combine multiple conditions using logical operators (& for AND, | for OR).

```
# Filter rows where Age is greater than 25 and Salary is less than 60000

filtered_df = df[(df['Age'] > 25) & (df['Salary'] < 60000)]

print(filtered_df)
```

2. Adding/Modifying Columns

Pandas allows you to create new columns or modify existing ones easily.

- **Adding a New Column**: You can create a new column by assigning values based on existing columns.

```
# Add a new column 'Tax' which is 10% of Salary

df['Tax'] = df['Salary'] * 0.10

print(df)
```

- **Modifying an Existing Column**: You can update values in an existing column by performing operations on it.

```
# Increase the Salary by 5%

df['Salary'] = df['Salary'] * 1.05

print(df)
```

- **Using apply() to Modify Columns**: The apply() function allows you to apply a function to a DataFrame column.

```
# Define a function to categorize ages

def categorize_age(age):

    if age < 30:

        return 'Young’

    else:

        return 'Old’


# Apply the function to create a new column 'Age Group’

df['Age Group'] = df['Age'].apply(categorize_age)

print(df)
```

3. Group By

Grouping data is essential for aggregation and summary statistics.

- **Group By for Aggregation**: The groupby() method allows you to group data by one or more columns and then apply aggregation functions like sum(), mean(), count(), etc.

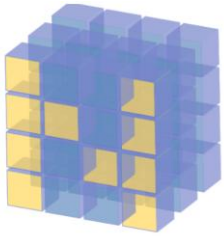
```
# Group by 'Age Group' and calculate the average Salary
group_df = df.groupby('Age Group')['Salary'].mean().reset_index()
print(group_df)
```

- **Multiple Aggregations**: You can perform multiple aggregation operations using the agg() method.

```
# Group by 'Age Group' and get multiple statistics
agg_df = df.groupby('Age Group').agg({ 'Salary': ['mean', 'max', 'min'], 'Age': 'count' }).reset_index()
print(agg_df)
```

3.9 NumPy Library

NumPy, short for Numerical Python, is one of the core libraries in Python for numerical computing. It's a foundational package in the data science ecosystem, widely used for performing efficient and fast computations on large datasets. NumPy is designed to handle multi-dimensional arrays and matrices efficiently, which makes it essential for data analysis, machine learning, and scientific computing.



Source: https://www.scaler.com/topics/architecture-of-data-warehouse/

3.9.1 Key Features of NumPy:

- **Efficient Handling of Numerical Data**:
  - NumPy is optimized for operations on large arrays of data, which are stored in contiguous memory blocks. This setup allows for quick data access and computation, making NumPy much faster for numerical tasks compared to standard Python lists.
- **Support for Multi-dimensional Arrays**:
  - The array is the central data structure in NumPy, and it can be one-dimensional (vectors), two-dimensional (matrices), or multi-dimensional (tensors). This versatility makes NumPy suitable for complex mathematical operations and multi-dimensional data manipulation.
- **Comprehensive Mathematical Functions**:
  - NumPy provides a broad range of mathematical functions for array operations, including linear algebra, statistical functions, random sampling, and more. These functions make it easy to perform a wide array of calculations, from basic arithmetic to advanced linear algebra operations.

- **Essential for Scientific Computing and Data Analysis**:
    - Due to its speed and efficiency, NumPy serves as the backbone of many other Python libraries for scientific computing and data analysis, such as Pandas, Matplotlib, and SciPy. Together, they form a powerful ecosystem for analyzing, visualizing, and working with large-scale data.

3.9.2 NumPy Array

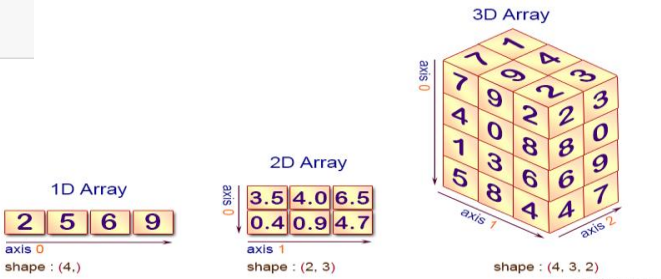
- The NumPy array is a multi-dimensional data structure that serves as the foundation for scientific computing in Python. Arrays are the primary way of storing and manipulating numerical data in NumPy, and they are highly optimized for performance, making them essential in data analysis and machine learning.

Key Characteristics of NumPy Arrays:

- **Homogenous Data**: All values within a NumPy array are homogenous, meaning they are of the same data type (e.g., all integers, all floats, etc.). This uniformity makes processing faster and more memory-efficient, as operations can be applied uniformly across all elements.
- **Multi-Dimensional Structure**: Unlike traditional lists in Python, NumPy arrays can be multi-dimensional. This allows you to create arrays of any shape (1D vectors, 2D matrices, or higher-dimensional tensors), making them versatile and powerful for representing complex data structures, such as images or time-series data.
- **Automatic Vectorization and Broadcasting**:
    - **Vectorization**: NumPy arrays support automatic vectorization, meaning you can perform mathematical operations on entire arrays without explicit loops. This leads to code that is not only faster but also more concise and readable.
    - **Broadcasting**: NumPy's broadcasting feature enables operations on arrays of different shapes, which simplifies many mathematical operations. For example, you can add a scalar value to an array, or add two arrays of different shapes, without extra code.

```python
import numpy as np
a = np.array([0, 1, 2, 3])        # Create a rank 1 array
print(a)                          #print array a
print(type(a))                    #type of array a
print(a.ndim)                     #dimension of array a
print(a.shape)                    #shape(row,column) of array a
print(len(a))                     #length of array a
```

```
[0 1 2 3]
<class 'numpy.ndarray'>
1
(4,)
4
```



Explanation of the code:

- This code begins by importing the NumPy library as np, which is a popular library in Python for numerical computations.
- It then creates a one-dimensional (rank 1) array a containing the integers 0 through 3.

- The code proceeds to print the array a itself, displays its type (which will confirm it's a NumPy array), and checks the number of dimensions using a.ndim, which should return 1 for a one-dimensional array.
- Additionally, it retrieves and prints the shape of the array with a.shape, which indicates the size of each dimension; since a is one-dimensional, this will return a tuple with a single value representing the length of the array.
- Finally, it prints the length of the array using the built-in len() function, which confirms the number of elements in a, returning 4 in this case.

3.9.3 Working With NumPy:

Creating a 2-D Array from a List:

- The code first imports the NumPy library as np. It then defines a list called mylist, which is a nested list containing three sublists: [1, 2, 3], [4, 5, 6], and [7, 8, 9].
- The np.array(mylist) function is used to convert this nested list into a 2-D NumPy array. This results in a 2-D array with three rows and three columns.

```
# Create 2-D array using numpy from list
import numpy as np
mylist=[[1,2,3],[4,5,6],[7,8,9]]
np.array(mylist)

✓ 0.0s
```

```
array([[1, 2, 3],
       [4, 5, 6],
       [7, 8, 9]])
```

Creating Random Arrays:

- **np.random.rand(5):** This function generates a 1-D array of 5 random values drawn from a uniform distribution between 0 and 1.
- **np.random.rand(4, 4):** This generates a 2-D array with 4 rows and 4 columns, filled with random values from a uniform distribution between 0 and 1.
- **np.random.randn(4):** This creates a 1-D array of 4 random values drawn from a standard normal distribution (mean 0 and standard deviation 1).

3.9.4 Commonly Used NumPy Methods

NumPy provides a variety of methods that are essential for creating and manipulating arrays. Here are some of the most commonly used functions:

- **np.array()**:
    - This function creates a **NumPy array** from a list or any other iterable. It's the fundamental method to convert standard Python data structures into NumPy arrays, enabling all the advantages of array operations.

- **Example**: np.array([1, 2, 3]) creates a one-dimensional array with three elements.

- **np.random.randint(0, 10, 1)**:
  - Generates a **random integer** between 0 and 10, with the specified size. This method is useful for simulations and random sampling.
  - **Example**: np.random.randint(0, 10, 1) might produce an output like array([4]).
- **np.unique()**:
  - Finds the **unique elements** in an array and returns them in sorted order. This is particularly useful for data analysis when you want to identify distinct values.
  - **Example**: np.unique([1, 2, 2, 3]) would return array([1, 2, 3]).
- **np.dot()**:
  - Computes the **dot product** of two arrays or matrices. This is fundamental in linear algebra for various applications, including machine learning algorithms.
  - **Example**: np.dot(np.array([1, 2]), np.array([3, 4])) yields 11, which is calculated as 1∗3+2∗41*3 + 2*41∗3+2∗4.
- **np.power(a, 2)**:
  - Raises each element of the array a to the power of 2. This is a convenient way to apply element-wise operations on arrays.
  - **Example**: If a = np.array([1, 2, 3]), then np.power(a, 2) results in array([1, 4, 9]).
- **np.sqrt(a)**:
  - Returns the **square root** of each element in the array a. This is useful for statistical analysis and data transformation.
  - **Example**: For a = np.array([1, 4, 9]), np.sqrt(a) would return array([1., 2., 3.]).
- **np.ones()**:
  - Creates a new array of the specified shape, filled with **ones**. This can be useful for initializing matrices or tensors in computations.
  - **Example**: np.ones((2, 3)) creates a 2x3 array filled with ones.
- **np.zeros()**:
  - Similar to np.ones(), this method creates a new array of the specified shape, but filled with **zeros**. It's often used to initialize arrays before populating them with data.
  - **Example**: np.zeros((2, 3)) generates a 2x3 array filled with zeros.
- **np.max()**:
  - Returns the **maximum value** in the array. This is helpful for identifying the largest element in datasets.

- **Example**: For a = np.array([1, 5, 3]), np.max(a) would return 5.

- **np.arange(10)**:
  - Creates an array with **evenly spaced values** from 0 to 9. It's similar to Python's built-in range() function but returns a NumPy array instead.
  - **Example**: np.arange(10) results in array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9]).

3.10 Matplotlib

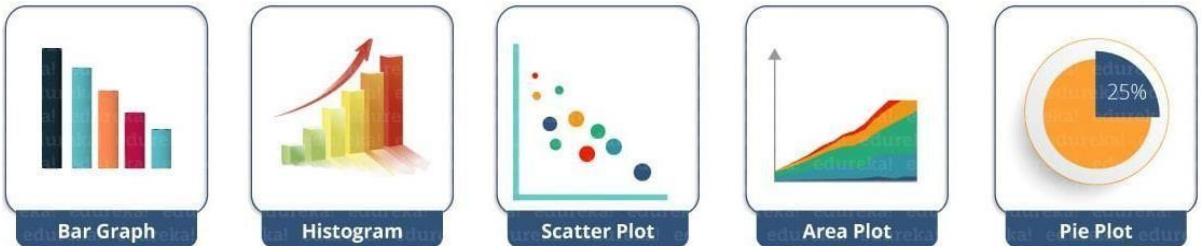
- Matplotlib is a powerful plotting library for Python that enables users to create a wide array of visualizations, including static, animated, and interactive plots. It is an essential tool for data analysis and presentation, allowing you to visualize data in an informative and aesthetically pleasing manner. Its flexibility and customization options make it a preferred choice for data scientists and analysts alike.

Source: https://www.scaler.com/topics/architecture-of-data-warehouse/

- **Highly Customizable and Versatile**: Matplotlib provides a comprehensive suite of tools and options that allow users to customize every aspect of their plots, from the overall layout to individual data point styling. Whether you need simple line graphs or complex multi-dimensional visualizations, Matplotlib has the capabilities to accommodate your needs.

3.10.1 Key Features of Matplotlib

- **Wide Range of Plots**:
  - Matplotlib supports various types of plots to visualize data effectively, including:
    - **Line Plots**: Ideal for displaying trends over time or continuous data.
    - **Bar Charts**: Useful for comparing categorical data or showing quantities.
    - **Scatter Plots**: Effective for visualizing relationships between two variables, showcasing correlations, and identifying outliers.
    - **Histograms**: Great for showing the distribution of numerical data and understanding the frequency of values within specific ranges.
    - **Box Plots**: Helpful for summarizing the distribution of data through their quartiles, showcasing the median, and identifying potential outliers.



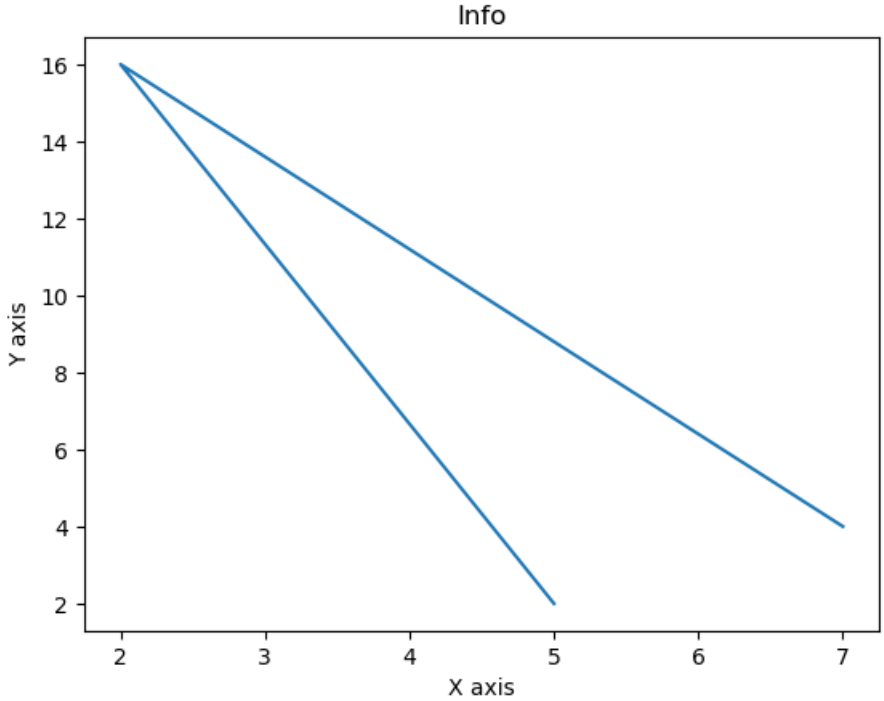
Source: https://www.edureka.co/blog/how-to-become-a-python-developer/

- **Control Over Plot Appearance**:
  - Matplotlib allows for extensive customization of plot aesthetics, enabling users to enhance the visual appeal and clarity of their visualizations. Key aspects include:
    - **Colors**: Users can choose from a wide palette of colors for lines, markers, and fills, allowing for the distinction between different data series or categories.
    - **Labels**: The ability to add informative axis labels, titles, and legends improves interpretability and ensures that the audience understands the context of the visualization.
    - **Scales**: Matplotlib supports various scales (linear, logarithmic, etc.) for axes, allowing for more meaningful representations of data, especially when dealing with a wide range of values.
    - **Annotations**: Users can annotate specific points on their plots to provide additional information or highlight important trends or data points.

Example of a Matplotlib plot:

```python
from matplotlib import pyplot as plt
x = [5,2,7]
y = [2,16,4]
plt.plot(x,y)
plt.title('Info')
plt.ylabel('Y axis')
plt.xlabel('X axis')
plt.show()
```

✓ 2.0s



3.10.2 Commonly Used matplotlib.pyplot Functions

- Matplotlib's pyplot module provides a simple interface for creating a variety of visualizations in Python. Below are some of the most commonly used functions, along with their purposes and examples of use.
- **plot()**:

  This function is used to plot lines or markers on axes. It's the most basic and versatile plotting function in Matplotlib.

  Example:

  ```
  import matplotlib.pyplot as plt

  x = [1, 2, 3, 4]

  y = [10, 20, 25, 30]

  plt.plot(x, y, marker='o') # Plots a line graph with circular markers at each point

  plt.title('Line Plot Example')

  plt.show()
  ```

- **scatter()**:

  Creates a scatter plot, which displays values for two variables as points in a Cartesian plane.

  Example:

  ```
  import matplotlib.pyplot as plt
  ```

```
x = [1, 2, 3, 4]
y = [10, 20, 25, 30]
plt.scatter(x, y, color='blue') # Plots a scatter plot of x vs y
plt.title('Scatter Plot Example’)
plt.xlabel('X-axis’)
plt.ylabel('Y-axis’)
plt.show()
```

- **bar()**:

  Generates a bar graph to compare different categories or groups of data.

  Example:

```
import matplotlib.pyplot as plt
categories = ['A', 'B', 'C’]
values = [3, 7, 5]
plt.bar(categories, values, color='orange') # Creates a vertical bar graph
plt.title('Bar Graph Example’)
plt.show()
```

- **barh()**:

  Similar to bar(), but creates a horizontal bar graph, which can be useful for displaying long category names.

  Example:

```
import matplotlib.pyplot as plt
categories = ['A', 'B', 'C’]
values = [3, 7, 5]
plt.barh(categories, values, color='green') # Creates a horizontal bar graph
plt.title('Horizontal Bar Graph Example’)
plt.show()
```

- **hist()**:

  Plots a histogram to visualize the distribution of a dataset by grouping data into bins.

  Example:

```
import matplotlib.pyplot as plt

data = [1, 2, 2, 3, 3, 3, 4, 4, 5]

plt.hist(data, bins=5, color='purple', alpha=0.7) # Creates a histogram with 5 bins

plt.title('Histogram Example')

plt.show()
```

- **hist2d()**:

  Creates a 2D histogram plot, which is useful for visualizing the joint distribution of two variables.

  Example:

```
import numpy as np

x = np.random.randn(1000)

y = np.random.randn(1000)

plt.hist2d(x, y, bins=30, cmap='Blues') # Creates a 2D histogram

plt.title('2D Histogram Example')

plt.colorbar() # Adds a color bar to indicate frequency

plt.show()
```

- **boxplot()**:

  Generates a box and whisker plot, which summarizes the distribution of data based on a five-number summary: minimum, first quartile, median, third quartile, and maximum.

  Example:

```
import numpy as np

data = [np.random.normal(0, std, 100) for std in range(1, 4)]

plt.boxplot(data) # Creates a box plot for multiple datasets

plt.title('Box Plot Example')

plt.xticks([1, 2, 3], ['STD 1', 'STD 2', 'STD 3'])

plt.show()
```

- **pie()**:

  Plots a pie chart to display the proportions of a whole for different categories.

  Example:

```
import numpy as np

sizes = [15, 30, 45, 10] # Proportions of each category

labels = ['Category A', 'Category B', 'Category C', 'Category D']
```

```
plt.pie(sizes, labels=labels, autopct='%1.1f%%', startangle=140) # Creates a pie chart
plt.title('Pie Chart Example')
plt.axis('equal') # Equal aspect ratio ensures that pie chart is circular
plt.show()
```

3.11 Seaborn

- Seaborn is a data visualization library built on top of Matplotlib, specifically designed to create attractive, informative, and easy-to-interpret statistical graphics. It simplifies the process of generating advanced plots and brings built-in themes and color palettes, making visualizations more aesthetically pleasing with less effort.

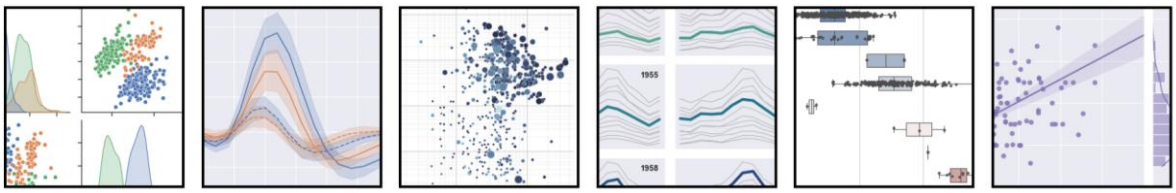
Source: https://seaborn.pydata.org/

- Seaborn provides a high-level interface for visualizing statistical relationships within data, making it especially useful for data analysis and exploratory data visualization. It is commonly used in data science workflows, especially for examining distributions, relationships, and comparisons within datasets.

3.11.1 Key Features of Seaborn

- **Built-in Themes**:
  - Seaborn offers a variety of themes (such as "darkgrid," "whitegrid," "dark," "white," and "ticks") that automatically improve the visual aesthetics of plots, making it easy to produce publication-quality graphics.
  - **Advantages**: These themes are fully customizable, allowing for quick adjustments to match specific styling needs without additional code or styling commands.
- **Easy Creation of Complex Visualizations**:
  - Seaborn simplifies the creation of complex, data-rich visualizations that might otherwise require extensive code in Matplotlib.
  - **Common Plot Types**:
    - **Pair Plots**: Plot pairwise relationships across multiple features in a dataset, useful for detecting patterns and correlations.
    - **Heatmaps**: Display matrix-like data with color-coding to represent values, commonly used to visualize correlation matrices.

- **Facet Grids**: Create multi-plot grids that display subsets of data, allowing for easy comparison between variables or categories.



Source: https://seaborn.pydata.org/

3.11.2 Basic Plot Types in Seaborn

- Seaborn offers a range of plot types to help visualize data quickly and effectively. These basic plot functions make it easy to interpret data, explore relationships, and communicate insights.

sns.scatterplot(): Creates a scatter plot to show the relationship between two continuous variables. It's ideal for exploring correlations and identifying patterns or clusters in data.

sns.lineplot(): Generates a line plot to illustrate trends in data over continuous intervals, such as time series or any ordered sequence.

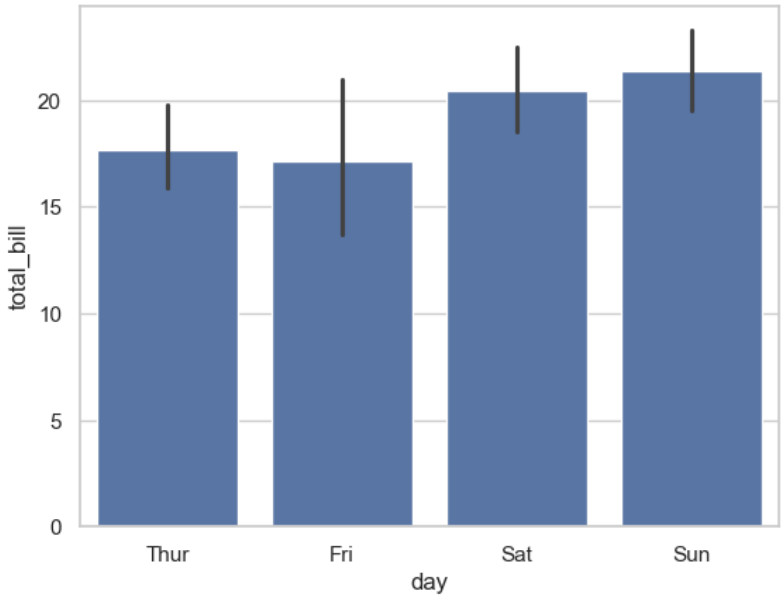
sns.barplot(): Draws a bar plot to compare values across categorical data. This plot is commonly used to display mean or sum values for categories.

sns.histplot(): Creates a histogram to visualize the distribution of a single continuous variable by grouping data into intervals (or bins).

Example of Seaborn plot:

```
import seaborn as sns
sns.set(style="whitegrid")
tips = sns.load_dataset("tips")
sns.barplot(x="day", y="total_bill", data=tips)
```

✓ 3.7s



3.11.3 Complex Plot Types in Seaborn

Seaborn's complex plot types are powerful tools for in-depth data analysis, particularly useful for visualizing relationships, distributions, and correlations across multiple variables.

sns.heatmap():

- Displays data in a matrix form, often used to visualize correlations between variables or patterns within a dataset.
- Heatmaps are frequently used for correlation matrices, where colors indicate the strength and direction of relationships.

sns.pairplot():

- Generates a grid of scatter plots showing pairwise relationships between numerical variables. It's a quick way to examine interactions and potential correlations between multiple variables.
- Pair plots are often used in exploratory data analysis (EDA) to spot patterns and relationships across many variables at once.

sns.violinplot():

- Combines features of a box plot and a KDE plot to show the distribution of a dataset across categories, indicating density.
- Violin plots are useful for comparing distributions between groups, especially to see if there's variability or concentration around specific values.

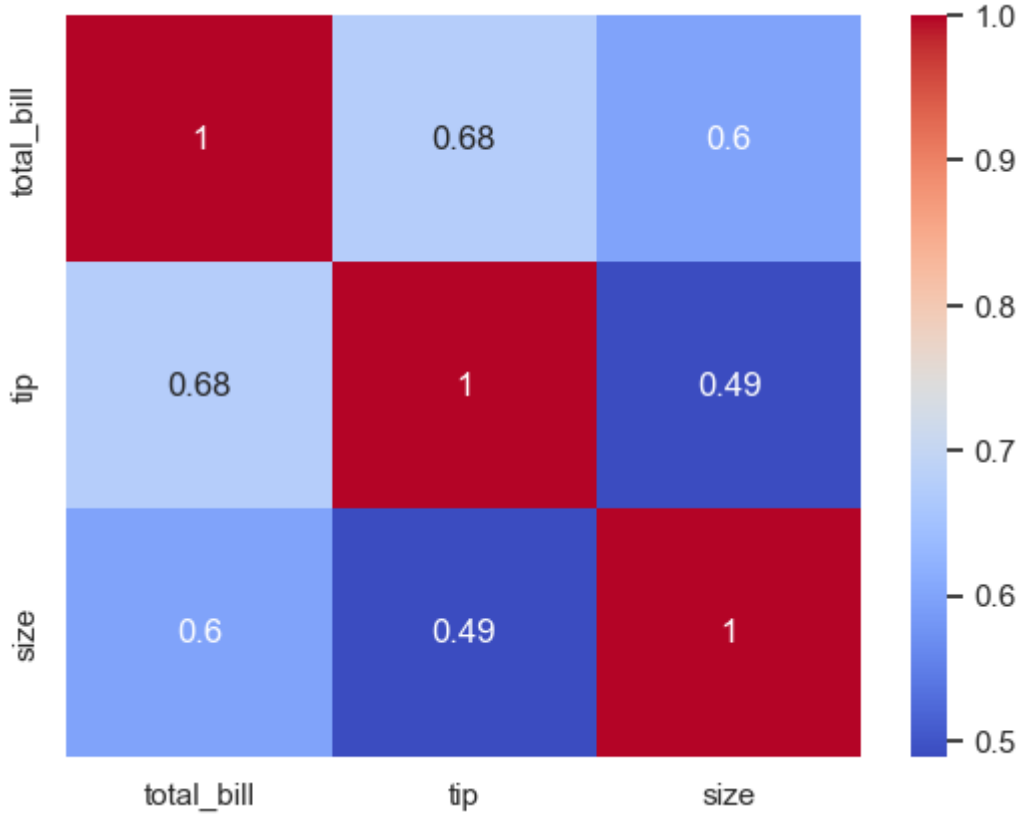
sns.regplot():

- Plots scatter plots with an optional linear regression line to examine the relationship between two variables.
- Regplots are used for trend analysis to see if there is a linear relationship between two variables.

Example of an Advance Seaborn Plot:

```python
# Example of a Advance Seaborn Plot:
import seaborn as sns
sns.set(style="whitegrid")
tips = sns.load_dataset("tips")
numeric_data = tips.select_dtypes(include='number')
sns.heatmap(numeric_data.corr(), annot=True, cmap="coolwarm")
```

✓ 0.2s



References:

- https://www.javatpoint.com/python-features
- https://www.geeksforgeeks.org/python-language-advantages-applications/
- https://docs.anaconda.com/anaconda/install/index.html
- https://www.w3schools.com/python/python_datatypes.asp
- https://realpython.com/python-conditional-statements/
- https://www.w3schools.com/python/python_classes.asp
- https://pandas.pydata.org/docs/getting_started/intro_tutorials/index.html
- https://numpy.org/doc/stable/user/quickstart.html
- https://matplotlib.org/stable/tutorials/introductory/pyplot.html